

# The LIBOR market model

Mark Joshi, Melbourne University

November 21, 2011

©Mark Joshi 2011

# Part I

Examples of products that can be priced using the LMM

# Forward rate agreement (FRA)

- This product pays the difference between a realized LIBOR rate and a strike.
- Pays at the end of an accrual period.
- For times  $t_1$  and  $t_2$  the pay-off is

$$(f - K)\tau P_2$$

- where  $f$  is the LIBOR rate,  $P_2$  is the discount bond expiring at  $t_2$  and

$$\tau = t_2 - t_1.$$

# Forward rates

- The forward rate is the rate which makes a FRA have zero value:

$$f = \frac{1}{\tau} (P_1/P_2 - 1).$$

- Note that the rate  $f$  and the ratio

$$P_1/P_2$$

contain the same information.

## Collection of rates

- For most LIBOR products, there will be a sequence of dates  $0 < t_0 < t_1 < \cdots < t_n$ .
- The coupon(s) at time  $t_j$  will depend on (many of) the forward rates between these dates:

$$f_k = \frac{1}{\tau_k} \left( \frac{P_k(t_j)}{P_{k+1}(t_j)} - 1 \right).$$

# Caplet

- This is an option on a forward rate agreement, we receive the positive part of its pay-off :

$$(f - K)_+ \tau P_2.$$

- If  $f$  is log-normal, it can be priced using the Black formula:

$$(fN(d_1) - KN(d_2))\tau P_2.$$

- Floorlet similar but

$$(K - f)_+ \tau P_2.$$

# In arrears caplet

- Pays

$$(f - K)_+ \tau P_1.$$

- So pays at the start of forward rate rather than the end.
- The Black formula does NOT hold.
- It is possible to develop a formula but its more complicated.

# Caps and floors

- Sum of caplets across adjacent periods.
- So, for example, a caplet on a forward rate agreement for each six month period for the next 10 years.
- Actually caps that are traded not caplets.
- Caplet prices and implied volatilities are generally interpolated from cap prices.
- Floorlets and floors similarly



# Flexi-cap

- Like a cap but can only be exercised for a small number of periods. E.g. 5 out of 20.
- Buyer chooses which periods.
- Price now depends on exercise strategy.
- Also, heavily dependent on interactions between different forward rates.

# Auto-cap

- Like flexi-cap but no choice.
- Buyer must exercise if caplet is in the money (or above some trigger.)
- No exercise strategy issues but product becomes discontinuous in the levels of rates.
- Still highly dependent on covariances of rates.

# Caption

- Option to buy a cap at some future time.
- Modelling will depend on both forward rates and forward volatilities.

# Fancy bonds

- Bank issues bond to investor.
- Bond pays fancy coupon, generally non-negative.
- Bond may or may not be callable.
- Or triggerable.
- Investor pays principal at beginning.
- Investor gets back principal at end.

## Bonds as swaps

- Bank regards bond as source of funding.
- So receives floating LIBOR rate (plus spread) on funding as long as bond lasts.
- So these products are typically regarded as swaps between a fancy coupon and a floating rate.
- Swap can be breakable by trigger or choice.

# Ratchet

- Coupon can only go up.

- So, e.g.

$$C_{i+1} = \max(C_i, f_{i+1}\tau_{i+1}).$$

- Lots of variants. e.g. snowball

# Inverse floater

- Coupon goes up when rates goes down
- Pays

$$(K - 2f_i)_+ \tau_i$$

at time  $t_i+1$ .

- Swap against floating rate.
- Really just floorlets so easy to price.
- The callable version is not so easy.

# TARN

- Targeted Range Note
- Big coupon first year, eg 12 %.
- Future coupon inverse floating possibly paid at  $t_i$  rather than  $t_{i+1}$ . i.e. in arrears
- Bond terminates when a fixed total coupon has been reached, eg 15%
- With upwards sloping yield curves, highly discontinuous since a slight shift can result in difference between a 2-year bond and a 10-year one paying no coupons.



# European swaption

- Right but not obligation to enter into a swap. A swap is worth

$$(SR - K) \sum_j \tau_j P_{j+1}.$$

- So the pay-off is the positive part of this.

# Bermudan swaption

- We have

$$0 < t_0 < t_1 < \cdots < t_n.$$

- We can enter in a swap on any *one* of these dates up to time  $t_n$ .
- So Bermudan option with pay-off:

$$(SR_i - K)_+ \sum_{j=i}^{n-1} \tau_j P_{j+1}$$

- with  $SR_i$  swap-rate starting at  $t_i$  and ending at  $t_n$ .

# Snowball

- Swap floating rate against path-dependent coupon:

$$C_{i+1} = (C_i + A_i \tau_i - f_i \tau_i)_+$$

- Cancellable.
- The rates  $A_i$  are increasing.
- Combination of cancellable and path-dependent!

# Trigger swap

- Swap comes into existence when a rate crosses a certain level.
- Similar to Bermudan swaption but exercise strategy is fixed rather than chosen.
- Can be very discontinuous or quite benign.

# CMS TARN

- Similar to the previous TARN but the coupon is a function of a swap-rate rather than a LIBOR.
- So it would, for example, always be the 10-year swap-rate.
- CMS = constant maturity swap

# What drives all these products?

- Most products appear to offer high investments yields: so exist for speculative reasons.
- The yield is generally at a price in terms of optionality or risk.
- Almost all exotic IRD products, the optionality belongs to the bank.

# How to classify?

- Do they depend on covariance of rates?
- Ability to early exercise/early terminate.
- Discontinuity of pay-off.
- Are they discrete?
- Dependence on rates before expiry?
- Explicit dependence on forward volatility

# No covariance?

- If product does not depend on covariance don't bother with market models.
- Eg cap, swap, caplet, European swaption.
- Most products do, however.



## Early exercise

- The ability to make a choice considerably complicates the analysis since we need to decide an exercise strategy.
- Even worse when a product can be exercised several times: flexi-cap.
- Typically use Monte Carlo so deciding a strategy is not easy.

# Discontinuous

- Generally arise when product is triggerable.
- A slight change in rates makes the difference between triggering and not triggering so can result in big changes in value.
- Generally easy to price but hard to Greek.
- Callability never causes discontinuities.

# Discreteness

- Market models rely on a product being a function of a finite number of observable rates on a finite number of dates.
- E.g. a barrier option on a LIBOR future would not fit into this framework.
- Some products, e.g., a callable range accrual are not natural for the LMM but can be jammed into it.

## Dependence on rates before expiry

- Some products' pay-offs only depend on the value of each forward rate on its own expiry.
- E.g. a ratchet on LIBOR rate
- These products can be priced more simply as we only need to sample from the final distribution of all the rates.
- Not true of most products, however.

# Forward volatility

- All complicated products depend to some extent on forward volatility since it affects covariance.
- But this will be particularly so if the product contains forward optionality
- Eg capton, flexi-cap, Bermudan derivative

## Part II

# Introducing the LIBOR Market Model

# Introduction to interest rate models (why LMM?)

- There are two basic paradigms in derivatives pricing
  - Principle of no arbitrage and dynamic replication
  - Extrapolation from observable prices to unobservable ones
- The ideal model should do both.

# Mathematical definition of arbitrage

- An arbitrage is a portfolio which is
  - Self-financing, i.e. no money is injected or extracted, but we can and buy and sell instruments provided excess cash is retained.
  - Of zero initial value
  - Of non-negative final value
  - Of positive value with probability greater than zero.
- Arbitrages have positive expected value.



# Principle of no arbitrage

- In a market in equilibrium, there should not exist any arbitrages.
- In practical terms, we assign the price of a new instrument so that its introduction does not create any arbitrage portfolios.

# Arbitrage and dynamic replication

- A self-financing portfolio *replicates* an instrument if it has the same value (i.e. generates the same cash-flows) as the instrument no matter what happens.
- Classical example: delta hedging a call option in the Black–Scholes world
- If an option can be replicated then its sole non-arbitrageable price is the price of the replication portfolio.
- But how do we know that price is OK?

# Arbitrage and equivalent measures

- Two measures are equivalent if
  - They assign probabilities to the same sets
  - They have the same sets of zero probability.
- This means that the set of arbitrage portfolios does not change if we pass to an equivalent measure.

# Arbitrage and martingales

- Important properties of martingales
  - Value is always equal to the expectation of its value at some future time.
  - This property is preserved by integration against a martingale in financial terms, a trading strategy in martingales is a martingale.
- A martingale portfolio is not an arbitrage portfolio since zero current value implies zero expectation

# Martingale measures

- A measure in which everything is a martingale has no arbitrage portfolios so all prices are non-arbitrageable.
- Unfortunately, the real market is not a martingale measure.  
Two problems:
  - Some assets always grow in value: e.g. money market account
  - Risk premia

# Martingale pricing

- First, we pick a numeraire, an asset (or self-financing portfolio) that always has positive value.
- We then value everything in terms of the numeraire. (i.e. divide by its price)
- We then pass to an equivalent measure in which the value of everything (in numeraires) is a martingale.
- We then define the price of any new instrument to be its expected value.

# Pricing equation

If  $D$  is the price of our derivative and  $N$  is the value of the numeraire then

$$D(0) = N(0)\mathbb{E}\left(\frac{D(T)}{N(T)}\right),$$

where  $D(T)/N(T)$  should be understood to contain all cash-flows generated by the product up to time  $T$  transformed into numeraires.

# Numeraire and the measure

- The measure makes the ratio of assets prices to the numeraire into martingales.
- If we pick a different numeraire then the measure will change as well.
- This has practical as well as theoretical effects



# No arbitrage and martingale pricing

- There are no arbitrages in the martingale measure.
- So there are no arbitrages in the real-world, since the measures are equivalent.
- So martingale pricing gives us a method of proving prices are not arbitrageable!
- Dynamic replication gives a way of showing that prices are arbitrageable!

# What does this mean for us?

- We have 3 levels of instruments:
  - the fundamental instruments we specify dynamics for. i.e. pick a martingale measure for;
  - the observable instruments we calibrate to;
  - the unobservable instruments we price.
- Classes of models differ in how they make the split between 1 and 2.

# Short rate models

- In a short-rate model we specify a process for the short-rate,

$$dr = \alpha(r, t)dt + \beta(r, t)dW_t,$$

- And this implies dynamics for the money market account,

$$m_t = e^{\int_0^t r(s)ds},$$

which we use as numeraire.

# Short rate models continued

- So all bonds, caplets, forwards are in the second level.
- Our choices of  $\alpha$  and  $\beta$  have to both calibrate to their prices and to imply dynamics for their future prices.
- The entire market's state is determined by the value of the short rate. i.e. the market is Markovian in  $r$ .

# Consequences of being Markovian in $r$

- Easy to implement on a lattice and quick.
- Hard to calibrate.
- Can get some closed-form formulas.
- Dynamics of the yield curve are simple.
- The set of possible yield curves attainable at any one time frame is one-dimensional:
  - this can simply be too limiting;
  - if the pay-off depends on the spread between two swap-rates, this spread will not vary much in the model.

# Market models

- Replace unobservable short rate by observable market rates, e.g. forward rates or swap-rates.
- Our level 1 instruments are now  $n$  zero-coupons bonds with  $n$  say 20 or 30 or 40.
- Our numeraire is one of these bonds.
- We specify dynamics for the forward rates or swap-rates directly.

## Consequences of working with rates directly

- Implementation on a lattice is hard if not impossible so must use Monte Carlo.
- Calibration to the discount curve and options on observed rates is automatic.
- Complex dynamics for yield curves occur naturally.
- The set of possible yield curves is high-dimensional at each time frame.

# Mathematical set-up of the model

- We have  $n + 1$  dates,

$$t_0 < t_1 < \cdots < t_j < \cdots < t_n.$$

- For each of these dates,  $t_j$ , we have a zero-coupon bond with value  $P_j(t)$  at time  $t$ .
- These zero-coupon bonds are our fundamental tradable instruments. (i.e. the level 1 instruments from the classification.)



# Assigning dynamics

- We assign dynamics to the forward rates or the swap rates not to the bonds directly.
- The values of  $n$  forward rates or swap-rates will, in general, determine all the bond-ratios

$$P_j/P_N$$

with  $P_N$  the numeraire. Note that we do not get the actual values of the bonds, just their ratios

# Sorts of market models

- We get a different market model for each subset of  $n$  rates chosen.
- Common choices:
  - LIBOR market model
  - Co-terminal swap-rate market model
  - Co-initial swap-rate market model
  - CMS market model

# LIBOR market model

- In the LIBOR market model, also known as BGM or BGM/J, we assign dynamics to forward rates,

$$f_j = \frac{1}{\tau_j} \left( \frac{P_j}{P_{j+1}} - 1 \right),$$

with  $\tau_j = t_{j+1} - t_j$ .

- Clearly, knowledge of the bond ratios determines the forwards, and vice versa

# Co-terminal swap-rate market model

- We assign dynamics to swap-rates with the same ending dates. i.e. to  $t_{j+1}$  with  $j = 0, \dots, n-1$ :

$$SR_{j,n} = \frac{P_j - P_n}{\sum_{i=j}^{n-1} \tau_i P_{i+1}}.$$

- Again (with a little work), these rates determine and are determined by the bond-ratios.

## Features of co-terminal model

- We assign dynamics to co-terminal swap-rates.
- This means that calibration to the bond ratios and to ATM swaptions on the coterminal swap-rates is automatic.
- Most natural for products with early exercise features, particularly Bermudan swaptions.

# Co-initial swap-rate market model

- Assign dynamics to rates with the same start date, with ending date  $t_j$  for  $j = 1, \dots, n$ ,

$$SR_{0,j} = \frac{P_0 - P_j}{\sum_{i=0}^{j-1} \tau_i P_{i+1}}.$$

- Again (with a little work), these rates determine and are determined by the bond-ratios.

## Features of co-initial model

- We assign dynamics to co-initial swap-rates.
- This means that calibration to the bond ratios and to ATM swaptions on the co-initial swap-rates is automatic.
- We can also assign correlation between these rates automatically.
- Most natural for products depending on multiple swap-rates with the same reset date, e.g. a spread option.

# Constant maturity swap-rate (CMS) market model

- Assign dynamics to swap - rates with the same tenor  $t$ ,  
 $j = 1, \dots, n$ ,

$$SR_{j,j+t} = \frac{P_j - P_{j+t}}{\sum_{i=j}^{j+t-1} \tau_i P_{i+1}}.$$

- We set  $j + t$  to be  $n$  if bigger than  $n$ .
- Again (with a little work), these rates determine and are determined by the bond-ratios.



# Features of CMS model

- We assign dynamics to CMS rates.
- This means that calibration to the bond ratios and to ATM swaptions on the CMS rates is automatic.
- We can also assign correlation between these rates automatically.
- Useful for products with path-dependent pay-offs depending on these rates. Eg CMS TARN.
- LMM and coterminal-swap-rate MM are actually special cases.

## Which model to use?

- In practice, almost all work is done with the LIBOR market model.
- It is possible to make it mimic the other models, and thus get their advantages without having to implement 4 different models.
- We will return to this when studying calibration.

## How does one price in a market model?

- We use Monte Carlo simulation.
- We pick a numeraire.
- We calculate the dynamics of the rates with that numeraire.
- We calibrate to reference instruments (independent of the numeraire).
- We average discounted generated cash-flows across many paths.

## How to do one path.

- Evolve all of the rates to each price sensitive date (from the term sheet)
- At each date, work out what cash-flows would be generated by the product for that yield curve.
- Convert these cash-flows into units of the numeraire portfolio.
- At the end of the path, multiply the total number of numeraires generated by the numeraire's initial value.

# The numeraire portfolio

- We can use any bond (or positive linear combination of bonds as numeraire.)
- But we have an issue when we reach its maturity date.
- We think of the numeraire portfolio as a trading strategy.
- We can use a different bond for each step provided we keep the portfolio self-financing.

# Discretely compounding money market account

- Take  $P_0$  for first step.
- At time  $t_0$  get 1 unit of cash.
- Use this to buy

$$(1 + f_0(t_0)(t_1 - t_0))$$

units of  $P_1$ .

- At  $t_1$  buy

$$(1 + f_0(t_0)(t_1 - t_0))(1 + f_1(t_1)(t_2 - t_1))$$

units of  $P_2$  and so on.

## Part III

# Dynamics of forward rates in the LIBOR Market Model

# The dynamics of forward rates

- Our choice of dynamics has multiple constraints
  - 1 It should make the ratios of bonds to numeraires martingales.
  - 2 It should calibrate to chosen reference instruments, e.g. caplets.
  - 3 It should give plausible dynamics to the rates influenced by our views on the real-world behaviour.
  - 4 It shouldn't predict qualitative changes in the market without good reason



# Log-normality

- A starting point for dynamics is to take the rates to be log-normal:

$$\frac{df_j}{f_j} = \mu_j dt + \sigma_j dW_t^{(j)},$$

with the Brownian motions for different rates correlated.

- The volatility is chosen to get the caplet on the rate  $f_j$  correct.
- The drift to get the ratio of bond prices to the numeraire to be martingales.

# Drifts

- The ratio of the price of any asset to the numeraire's price will be a martingale and so have zero drift.
- Forward rates are not assets nor are they ratios of asset prices to the numeraire in general.
- Consequence: forwards are not driftless and the drifts are in general state-dependent

## Computing drifts

- The drifts have multiple effects.
- Time has to be spent computing them.
- Their value will vary according to the choice of numeraire.
- The rates are not truly log-normal and the SDE is not analytically solvable: this necessitates numerical approximations.

## General methodology for computing drifts for rates

- Let a rate,  $R$ , be a quantity that is defined as the ratio of two asset prices  $X$  and  $Y$ .
- It is the exchange rate between the two assets.
- E.g. for a forward rate

$$f_j = \frac{1}{\tau_j} \frac{P_j - P_{j+1}}{P_{j+1}}.$$

# Cross-variation derivative

- If we have two processes  $A$  and  $B$ ,

$$dA = \mu(A)dt + \sigma(A)dW_t^A,$$

$$dB = \mu(B)dt + \sigma(B)dW_t^B,$$

driven by correlated Brownian motions with correlation  $\rho$ .

- We define

$$\langle A, B \rangle = \sigma(A)\sigma(B)\rho.$$

# Properties of cross-variation derivative

- It is the coefficient of  $dt$  in  $dA.dB$ .
- Algebraic relations:

$$\langle A, B + C \rangle = \langle A, B \rangle + \langle A, C \rangle,$$

$$\langle A, B \rangle = \langle B, A \rangle,$$

$$\langle A, BC \rangle = \langle A, B \rangle C + \langle A, C \rangle B,$$

$$\langle A, 1 \rangle = 0,$$

$$\langle A^{-1}, B \rangle = - \frac{1}{A^2} \langle A, B \rangle.$$

## Cross-variations and drifts

- If  $N$  is the numeraire and  $R = X/Y$  then in the martingale measure, we have that

$$\frac{RY}{N} = \frac{X}{N} \text{ and } \frac{Y}{N},$$

are both martingales.

- By Ito's lemma

$$d\left(\frac{RY}{N}\right) = dR \cdot \frac{Y}{N} + R d\left(\frac{Y}{N}\right) + dR \cdot d\left(\frac{Y}{N}\right).$$

- The martingales have zero drifts, so ...

## Cross-variations and drifts II

- So, equating drifts:

$$0 = \frac{Y}{N} \mu_R + \langle R, Y/N \rangle.$$

- We get

$$\mu_R = -\frac{N}{Y} \langle R, Y/N \rangle.$$

- This is a very general result.



# The drifts of forwards

- We want to compute the drift of  $f_j$  and our numeraire is  $P_N$ .
- We suppose

$$df_j = \mu_j(f, t)dt + \sigma_j(f_j, t)dW_t^{(j)},$$

with the Brownian motions driving rates  $j$  and  $k$  having correlation  $\rho_{jk}$ .

## Drifts of forwards II

- We have

$$f_j = \frac{1}{\tau_j} \frac{P_j - P_{j+1}}{P_{j+1}},$$

- so

$$\mu_j = -\frac{P_N}{P_{j+1}} \langle f_j, P_{j+1}/P_N \rangle,$$

- and

$$P_{j+1}/P_N = \prod_{k=j+1}^{N-1} (1 + f_k \tau_k).$$

## Drifts of forwards III

- We compute

$$\langle f_j, \prod_{k=j+1}^{N-1} (1 + f_k \tau_k) \rangle = \sum_{k=j+1}^{N-1} \frac{1}{1 + f_k \tau_k} \frac{P_{j+1}}{P_N} \langle f_j, 1 + f_k \tau_k \rangle.$$

- So

$$\begin{aligned} \mu_j &= - \sum_{k=j+1}^{N-1} \frac{\tau_k}{1 + f_k \tau_k} \langle f_j, f_k \rangle, \\ &= - \sum_{k=j+1}^{N-1} \frac{\tau_k \sigma_k(f_k, t) \sigma_j(f_j, t) \rho_{jk}}{1 + f_k \tau_k}. \end{aligned}$$

## Drifts of forwards IV

- If we are using log-normal volatilities

$$\sigma(f_j, t) = \sigma_j(t)f_j.$$

- Then

$$\frac{\mu_j}{f_j} = - \sum_{k=j+1}^{N-1} \frac{\tau_k f_k \sigma_k(t) \sigma_j(t) \rho_{jk}}{1 + f_k \tau_k}.$$

# Numeraire shorter than rate

- If  $N \leq j$  then we get

$$\frac{\mu_j}{f_j} = \sum_{k=N}^j \frac{f_k \tau_k \sigma_k(t) \sigma_j(t) \rho_{jk}}{1 + f_k \tau_k}.$$

- Similar, but the
  - sign is positive
  - drift of  $f_j$  depends on  $f_j$ .

# Observations on drifts

- The drift is state-dependent and depends upon more than one rate so no chance of analytically solving the SDE.
- The correlations and volatilities only enter as

$$C_{jk} = \rho_{jk} \sigma_j \sigma_k,$$

- So it is only covariance that matters.
- General point: “convexity” = drift = covariance between rates cash-flows and discounting

# State-dependence

- We get two important effects from the state-dependence
  - We have to compute the drifts at every step of every simulation, this is time-consuming!
  - We have to approximate the evolution of our SDEs.
- We will come back to these.

## Part IV

# Practical implementation of the LIBOR market model



# Practical implementation

- Let's suppose we have calibrated our model.
- We want to price a derivative.
- We will use Monte Carlo simulation.
- How do we do this effectively?
- LMM is fast if done well, but can be very slow if the wrong choices are made.

# Issues

- State-dependence of drifts
  - Computation
  - Approximation
- Discounting
- Convergence
  - Low discrepancy numbers

# Structuring the problem

- A product's cash-flows will depend upon the values of the rates on a finite number of dates, typically a subset of the reset times of the forward rates.
- We need to evolve the rates between each of these dates.
- Our inputs to the evolution are
  - the covariance matrix of the logs of the rates between these dates,
  - the initial values of the rates.

# Generating covariances

- Ignoring drifts for now, we have that the change in the logs of the forward rates across a step  $(s, t)$  has a given covariance matrix  $C(s, t)$ , and that they are jointly normally distributed.
- We need a methodology for generating normals with a given covariance matrix

# Generating covariance

- If  $W$  is a vector of independent normal  $N(0, 1)$  random variables, then we set

$$Z = AW.$$

- The covariance of  $Z_j$  and  $Z_k$  is then

$$\mathbb{E}(\sum_{k,m} a_{jk} W_k a_{lm} W_m) = \sum_r a_{jr} a_{kr}.$$

# Pseudo-roots

- So  $Z$  has the covariance matrix

$$C = AA^t.$$

- $C$  will be an  $n$  by  $n$  matrix.
- But  $A$  need not be.
- Typically, we take  $A$  to be  $n$  by  $F$  with  $F = 1, 2, 3, 5$ .
- Reduced-factor model.

# Factors

- The number of factors in LMM is the number of random numbers we use per step.
- Not the number of driving Brownian motions.
- The LMM is not Markovian in the Brownian motions.

# Markovian factors

- When doing a short-rate model, the total number of attainable yield curves was one-dimensional at every time frame, because everything was a function of  $r$ .
- In an  $F$  factor LMM model, after  $k$  steps, we have  $kF$  dim. of possible yield curves.
- Rates are not functions of the underlying Brownians.
- This effect arises from both the drifts and the time-dependence of volatilities



# Naive implementation

- We draw  $F$  random numbers,  $W$ , per step  $(s, t)$  with covariance matrix  $C$  and pseudo-root  $A$ .
- We set  $Z = AW$ .
- We put

$$\log f_j(t) = \log f_j(s) - \frac{1}{2} C_{jj} + \mu_j(f(s)) + Z_j.$$

- with

$$\mu_j = - \sum_{k=j+1}^N C_{jk} \frac{f_k \tau_k}{1 + f_k \tau_k}.$$

# Problems with the naive implementation

- Must make extra steps because approximation of the SDE solution is poor.
- The computational complexity of one step is  $n$ -squared.
- The computational complexity of  $n$  steps is  $n$ -cubed.
- The error after  $m$  paths is  $\sigma m^{-1/2}$ .

# Improving the SDE approximation

- One simple solution is to use predictor-corrector (PC).
  - Evolve the rates to the end of the step
  - Compute drifts at the end of the step
  - Take drift to be average of beginning and end.
  - Re-evolve using the same random numbers.
- This is much more accurate, can do very long steps, e.g. 20 years with small error

# Iterative PC (IPC)

- Recall that if  $N$  is greater than  $j$ , then the drift of  $f_j$ , only depends on  $f_k$  for  $k > j$ .
- This means that we can do the rates one at a time working backwards if  $N = n$ .
- We use the corrected version of  $f_k$  when working out the drift of  $f_j$  instead of the predicted version.
- This is therefore more accurate.

# Martingale failures

- Whilst IPC is accurate, there is the defect that the bond ratios are not martingales in the discretized measure.
- They are only very good approximations to martingales.
- Glasserman-Zhao suggest an alternative approach.

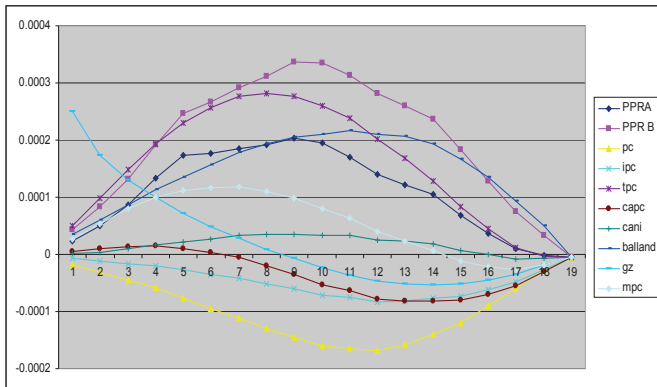
# Glasserman-Zhao

- Change coordinates and evolve bonds instead of rates.
- Main upside: bond ratios have zero drift.
- Main downside: volatilities are state dependent.
- Bond ratios are martingales after discretization so no discretization error for their prices.
- However, issues with positivity in spot measure.

# Other methods

- There are now about 10 methods.
- ipc and pc are good enough.
- Main variants:
  - Use geometric means (balland)
  - Use Brownian bridging to estimate drift along the path (PPR)
  - Use additional correlation information to estimate drifts at rate reset times during steps (cani capc)

Typical example 20 rates, yearly, one step, terminal measure, 20% vol:





## Forwards only on their own reset dates

- A pure LIBOR product which is non-callable has pay-off determined by forwards purely on their own reset dates.
- E.g. TARN pays
  - Fixed coupon first year
  - Inverse floating LIBOR coupon thereafter
  - Until fixed total coupon is reached
- Or swap triggering on a forward rate.

## Special features of pure LIBOR products

- The pay-off only depends on the joint terminal distribution of the LIBOR rates underlying the product.
- We can price with one step instead of stepping to each reset date.
- Sometimes called “very long step” procedure.
- Can be done in both spot and terminal measures: Chris Beveridge, “Very Long-Stepping in the Spot Measure of the LIBOR Market Model,” <http://ssrn.com/abstract=1444260>
- Better to work in spot measure since variances are lower.

## Very long stepping

- We compute the covariance matrix of all the log rates to the final reset time.
- Rates have zero vol after reset.
- We use a good drift approximation to evolve the rates in one step.
- We compute the generated cash-flows and discount.

## Very long stepping and factors

- Normally, when short stepping we reduce factors to reduce time taken.
- This is neither wise nor necessary when very long stepping.
- We are only doing one step so fast anyway.
- We do not get terminal decorrelation effects since one step, so must be careful not to kill decorrelation via factor reduction.

# Computational complexity

- For each step, we must
  - Compute the drifts (twice)
  - Multiply the Gaussians by the pseudo-root
  - Add
  - Compute cash-flows
- These have complexities
  - $n$ -squared
  - $nF$
  - $n$  for the last two

# Speeding up the drift computation

- If  $n$  large and  $F$  small, all the time will be spent on drifts.
- We want a faster drift computation method.
- The key is to rewrite the drifts in such a way that we can reuse computations.

# Rewriting the drift computation

- If  $N = n$ , (other cases are similar)
- Let  $g_k = \frac{f_k \tau_k}{1 + f_k \tau_k}$ ,
- Then

$$\begin{aligned}\mu_j &= - \sum_{k=j+1}^n \sum_{l=1}^F A_{jl} A_{kl} g_k, \\ &= - \sum_{l=1}^F A_{jl} \sum_{k=j+1}^n A_{kl} g_k,\end{aligned}$$

## Reusing information

- The value of the inner summand does not depend on  $j$ , and the sum can be computed for all values of  $j$  and  $l$  with order  $nF$  computations.
- Another order  $nF$  computations does the outer sums.
- So the total computation is of order  $nF$  instead of  $n$ -squared.
- If  $n$  is very large, this approach is essential.
- Also important for Greek computations.



# Numeraire choice

- In a theoretical sense, it makes no difference: the price is numeraire independent.
- In a practical sense, it does.
- The most popular choices are
  - $N = n$ , terminal measure
  - $N$  = shortest bond not yet expired. Discretely compounding money market account or spot measure.

## Terminal measure

- Drifts are all negative. This stops feedback leading to rate and drift blow-up when volatilities are high and rates are highly correlated.
- Same bond every step simplifies implementation.
- Can give high variance. This is because high rates mean that the numeraire is of very low value. The ratio of the cash-flow value to numeraire value is big.

## Discrete money market measure (spot measure)

- Drifts are all positive so feedback can lead to rate and drift blow-up when volatilities are high and rates are highly correlated.
- We use a different bond for each step
- Our numeraire is a portfolio of bonds,
- We sell each bond as it expires and use the value to buy a number of bonds with the next expiry.

## Discrete money market measure II

- Lower variance
- The numeraire value is always at least one
- No problems with the ratio of cash-flow value to numeraire value being large.
- Note other choices are possible (but uncommon) E.g use a bond in the middle

# Discounting in market models

The key is to realize that everything should be done in financial terms.

- The numeraire is a self-financing portfolio of bonds whose principal will change if we use different bonds for each step.
- Each cash-flow as it arrives is used to buy (or sell) units of the numeraire portfolio.
- Our value for the path is the total number of numeraire portfolios held at the end of the path.
- We multiply this by the initial numeraire value.

# Monte Carlo convergence

Monte Carlo simulation relies on two theorems from probability:

- Law of large numbers: the average of a large number of independent draws from a distribution converges to its expectation
- Central limit theorem: if  $\sigma$  is the standard deviation, the mean after  $m$  draws is approximately normal with standard deviation, called the standard error:  $\sigma/\sqrt{m}$ .

# Pros and cons of Monte Carlo

- We can measure standard deviation within the simulation so we know how accurate our result is.
- The convergence is order  $m^{-1/2}$ .
- The order is dimension independent but slow!
- Note the dimension of our integral is  $nF$ .
- Other integration methods tend to be faster in low dimensions and slower in high dimensions.

## Quasi-Monte Carlo

- One solution is to Quasi-Monte Carlo.
- Similar to Monte Carlo but input numbers are not drawn randomly.
- Instead they are chosen to fill out space uniformly.
- Examples of low-discrepancy numbers are: Sobol, Niederreiter, Faure.
- Sobol most commonly used for pricing.



# The use of Sobol

- Sobol numbers can be very effective but there are a number of implementation details which if done incorrectly can make them dangerous.
- Approximate order of convergence

$$\frac{(\log n)^d}{n}.$$

- If  $d$  small, or *effectively small* then fast.

## Maximizing impact

- Sobol numbers deteriorate with dimension.
- So we make the lowest dimensions do the most work.
- In practical terms, this means using more sophisticated methods of turning normals into Brownian paths, and using the spectral-pseudo-square root not the Cholesky decomposition

# Incremental path generation

- Suppose we want to generate a Brownian motion at times  $1, 2, 3, 4, 5, \dots, T$
- Take normals  $U_j$  and set

$$\begin{aligned}W_0 &= 0, \\ W_{j+1} &= W_j + U_j.\end{aligned}$$

- $U_1$  only has a small impact.

# Brownian bridge path generation

- We use  $U_1$  to determine the final point, and fill in intermediate points:

$$W_T = \sqrt{T} U_1,$$
$$W_{T/2} = \alpha W_T + \beta U_2.$$

With  $\alpha$  and  $\beta$  so that the covariance matrix of  $W_{T/2}$  and  $W_T$  is

$$\begin{pmatrix} T/2 & T/2 \\ T/2 & T \end{pmatrix}.$$

## Brownian bridge continued

- We then continue filling in gaps.
- The crucial point being that if

$$t_1 < t_2 < t_3,$$

and  $U$  is a standard normal, we can put

$$W_{t_2} = W_{t_1} + \frac{t_2 - t_1}{t_3 - t_1}(W_{t_3} - W_{t_1}) + \sqrt{\frac{t_2 - t_1}{t_3 - t_1}(t_3 - t_2)}U.$$

# Using the BB

- We start with a vector of Sobol numbers that are constructed from  $N(0, 1)$  distribution.
- We use these to generate a path with integer date spacings.
- We then take successive differences to get a vector of standard normals,  $N(0, 1)$ .
- Note for ordinary random numbers, all this should have zero effect.

# Pseudo-roots

- Given a matrix  $C$ , there are many pseudo-square roots,  $A$ , such that

$$C = AA^t.$$

- We want to choose the one that maximizes the impact of the lowest dimensions.

# Spectral-pseudo-root

- The covariance matrix,  $C$ , is a (semi-) positive definite symmetric matrix.
- It therefore has a complete basis of eigenvectors,  $e_i$ , and eigenvalues

$$\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_n$$

- For maximum impact, we set

$$A = \left( \lambda_1^{1/2} e_1 \mid \lambda_2^{1/2} e_2 \mid \dots \mid \lambda_n^{1/2} e_n \right).$$



# Steps versus factors

- We generally have  $n$  steps and need  $F$  random variables per step.
- We have to divide up our total of  $nF$  numbers.
- Three approaches
  - 1 Use first  $n$  for first factor, second  $n$  second factor
  - 2 Use first  $F$ , one for each factor, and so on
  - 3 Triangular method

# Triangular method

- We assign the variates as a compromise between two extreme methods
- Variates map to factors as follows:

$$1 \mapsto 1,$$

$$2 \mapsto 2,$$

$$3 \mapsto 1,$$

$$4 \mapsto 2,$$

$$5 \mapsto 3 \dots$$

# Dimensionality

- We need a certain number of quasi-random numbers per path:
- $d = \text{number of steps} \times \text{number of factors}$
- We must initialize our Sobol generator to this dimension as it is set up to do  $d$ -dimensional integrals.
- We draw a  $d$ -vector.
- We cannot draw numbers one at a time.

# Conversion to normals

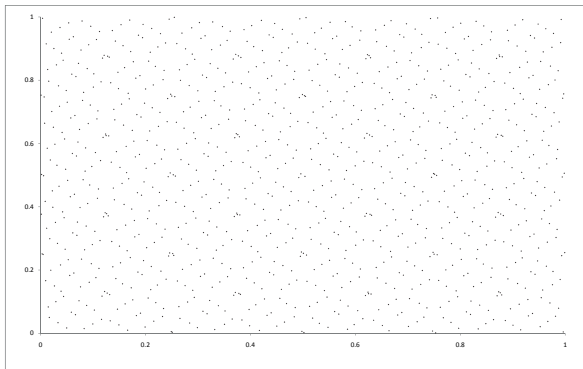
- A Sobol generator gives us a vector of  $d$  uniforms.
- We want  $d$  normals from  $N(0,1)$ .
- We cannot use standard methods such as
  - Acceptance-rejection: needs extra variates
  - Box-Muller: scrambles dimensions
- Best method is to use an approximation to the inverse cumulative normal.

# Generator initialization

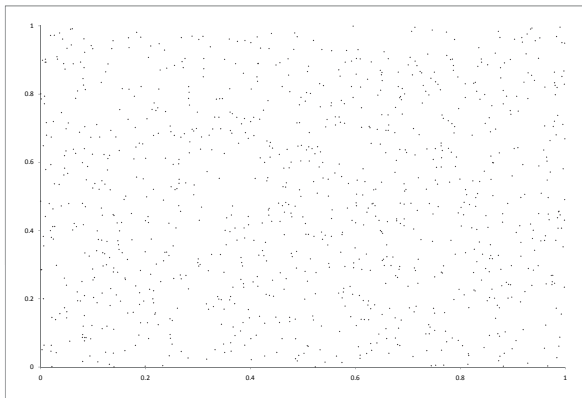
- For each dimension one needs to specify seeds.
- Common choices:
  - Unit - this is bad! (Numerical Recipes!)
  - Random
  - Chosen low dims, Bratley-Fox, Jackel, Joe-Kuo, Sobol-Levitan, Sobol-Levitan-Lemieux
  - Mix of chosen and random.

We will now various choices in good and bad 2-dim projections.  
We also look at Mersenne pseudo-randoms for comparison.

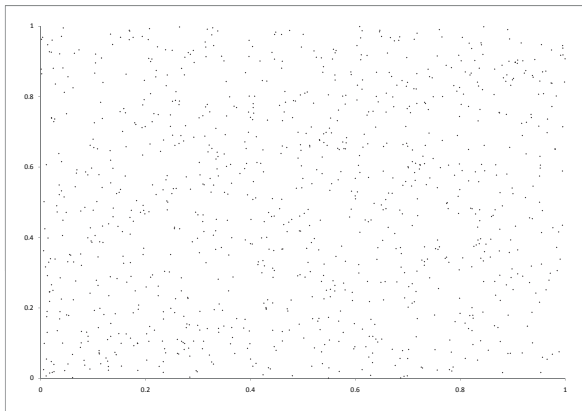
# Joe-Kuo D7 first two dimensions



# Mersenne first two dimensions

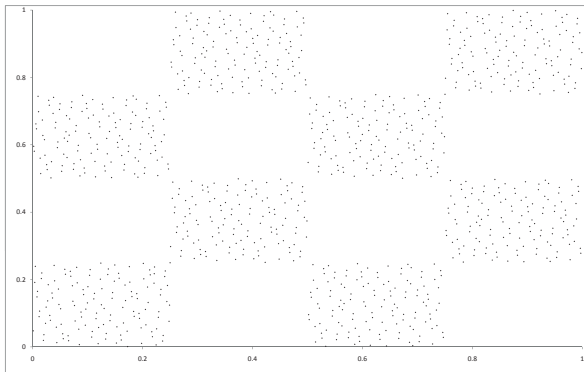


# Mersenne worst two dimensions

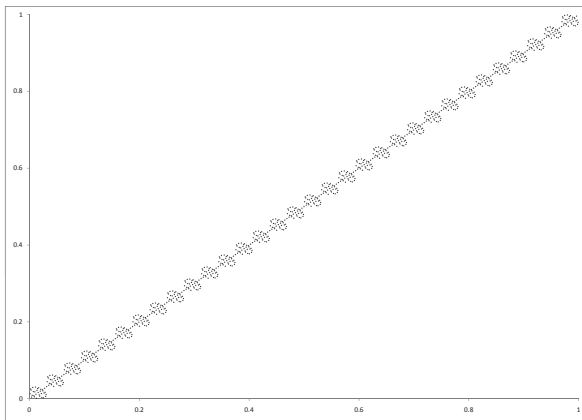




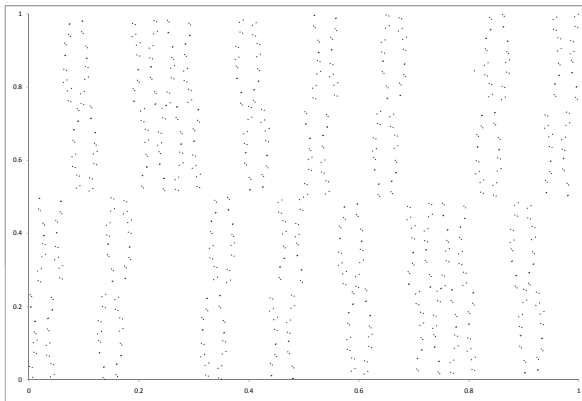
# Jäckel worst two dimensions



# Unit worst two dimensions



## Joe-Kuo D7 worst two dimensions



# Number of paths

- The first dimension does most work and follows the sequence:

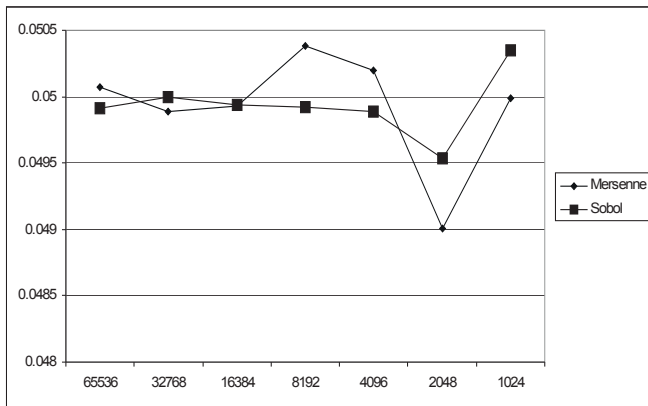
$$1/2, 1/4, 3/4, 1/8, 3/8, 5/8, 7/8, \dots$$

- This means that the integral is most converged after  $2^n - 1$  paths.
- Convergence will actually get worse if numbers in between are used.

## Error measures

- Main downside of QMC is that standard error does not reflect the errors.
- In fact, the in-sample estimate of standard error will be (up to error) the same as for MC.
- So if QMC is any good, this will be way too high.
- One solution is convergence diagrams.
- Another is randomized QMC.

# Convergence table for Bermudan, Sobol versus Mersenne



# Randomized QMC

- Compromise between MC and QMC
- If we draw a pseudo-random vector,  $x$ , of uniforms and add it modulo one to all of  $n$  paths from a low-discrepancy generator, then this gives a random estimate of the integral.
- If we do this  $M$  times, we have an  $M$  point Monte Carlo so can compute standard error. Uses total of  $Mn$  paths. Eg  $M = 16, n = 1024$ .

# Scrambled QMC

- Flaw of RMC: Sobol is designed to make moments correct for  $2^n - 1$  paths. Doing a random mean shift will kill this property.
- Alternate approach: draw random 32 bit integers and XOR the draws before converting to uniforms.



# Testing Randomized QMC

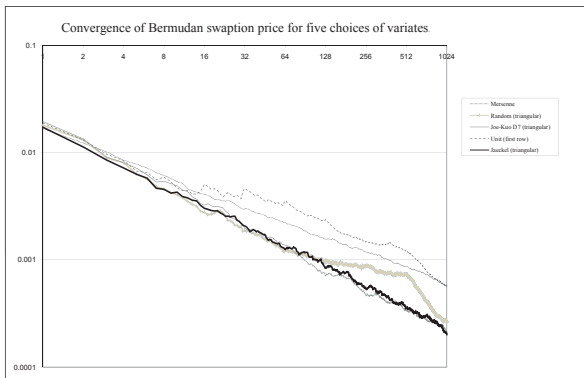
- Take  $N$  paths.
- Run 16 randomized simulations to get standard error.
- Do this many times.
- Take average standard error.

## Choices: init numbers

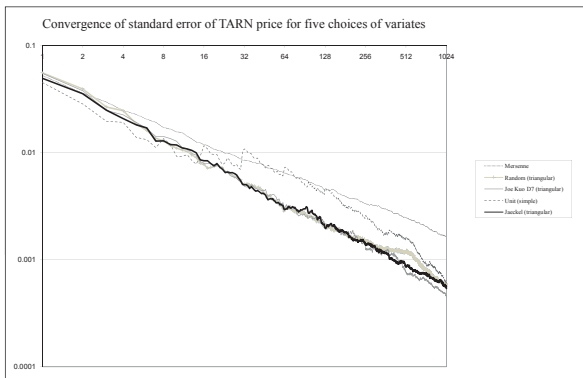
- Joe Kuo D7
- Unit
- Random
- Mersenne

9 underlying rates. 9 factor model. So dimensionality about 45.

# Convergence graph for Bermudan swaption



# Convergence graph for TARN



## References

- Glasserman, Monte Carlo Methods in Financial Engineering, 2002
- Hunter-Jäckel-Joshi, Getting the drift, Risk Magazine 2001
- Jäckel, Monte Carlo Methods in Finance, 2001
- Joshi, Rapid Computation of Drifts in the LIBOR Market Model, Wilmott 2003
- Joshi-Stacey, New and Robust Drift Approximations for the Libor Market Model, Quantitative Finance

## Part V

# Calibrating the LIBOR market model

## Simplistic calibration

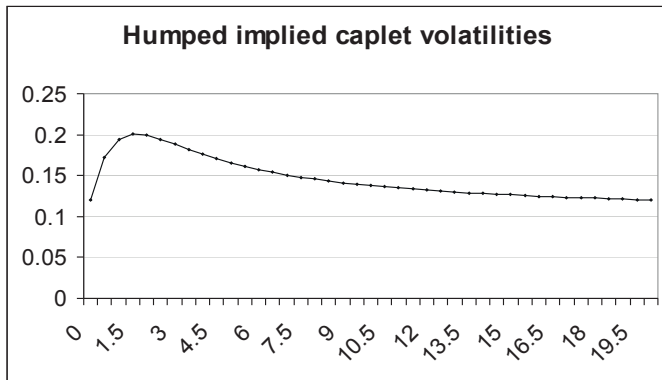
- The LIBOR market model was introduced because it allowed automatic and trivial calibration to the yield curve and caplet volatilities.
- Initial values of forward rates = market observed values.
- Log-normal volatilities = implied volatilities of caplets on these rates.
- And we are done

## Problems with simplistic calibration

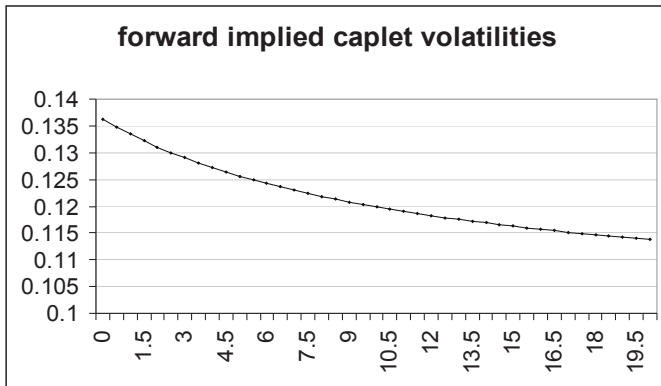
- We still have to decide the correlation between rates.
- Only caplets are priced correctly.
- Time inhomogeneity – the predicted future caplet market will not look like today's market.



# Caplet curve



# Time inhomogeneous



# What is a calibration?

- We evolve the rates from time step to time step.
- We only look at their values at these points.
- Their behaviour is determined by the pseudo-root of the covariance matrix for that step.
- So a calibration is precisely a choice of pseudo-root for each time step.

# Advantages of Pseudo-roots

- It's hard to keep control of the number of factors if we work with a covariance matrix.
- Pseudo-roots are always valid.
- Covariance matrices have to be positive semi-definite; this can be a pain to ensure.
- Main cons:
  - Possibly less intuitive
  - Price constraints are quadratic rather than linear.

# How many calibrations are there?

- If we use an  $F$ -factor model, and have  $n$  rates, then the pseudo-root for each step has  $nF$  entries.
- We typically have  $n$  steps, so we have

$$n^2 F$$

parameters.

- Symmetries and rates-setting reduces the number but this is the order; pseudo-roots are not unique.

# Imposing constraints

- We have a pseudo-root  $A_i$  for step  $i$ , and it has entries

$$A_{jk}^{(i)}$$

- $j$  denotes the rate,  $k$  the factor.
- I will assume that step  $i$  finishes at the reset time of rate  $i$ , since this is most typical.

# Not Enough

- If we specify the caplet on every rate, we get  $n$  constraints.  
So we have

$$n^2 F - n$$

constraints left.

## Time homogeneity

- A model should not predict a qualitative change in the market without good reason.
- For a long period of time, caplet implied vols have displayed a humped shape.
- They go up to about two years and then go down again.
- Our model should retain this property.
- Flat vols predicts that it disappears.



# Concepts of time

There are at least 3 concepts of time in the LMM and it is important to distinguish between them.

- Reset time, the time a rate starts:  $T_i$ .
- Calendar time, the current time along the path:  $t$ .
- Time to go: the time until a rate resets:  $T_i - t$ .

## Musiela parametrization

- Making everything a function of time to go is called the Musiela parametrization.
- This automatically gives time homogeneity: the implied vols in the market will look the same in the future as today.

# Time homogeneity

- We can impose the constraint that a forward rate's behaviour only depends upon the time until its reset not on calendar time,  $t$ , or on its maturity date,  $T$ , alone.
- i.e. its loadings are functions of  $T-t$ .

# Time homogeneity and pseudo-roots

- On the second step, the second rate behaves as the first rate did on the first step:
- This can be translated into

$$A_{jk}^{(l)} = A_{j-1,k}^{(l-1)}, \text{ zero off matrix.}$$

# The effect of time homogeneity

- The value of  $A^{(0)}$  determines all the other pseudo-roots.
- So we are down to  $nF$  entries to determine.
- If  $F = 1$ , i.e. perfect instantaneous correlation, caplet prices + time homogeneity determine the entire calibration.

# The limits of time homogeneity

- Not always possible.
- It requires that the variance of log forward  $i$  is increasing with  $i$ . i.e. that

$$\hat{\sigma}_i^2 t_i$$

is an increasing function of  $i$ .

- Failure can suggest that rates are not log-normal as much as the fact that the market is not time homogeneous.

# Decoupling

- One approach to the calibration problem is to divide the covariances into standard deviations (i.e. vols) and correlations.
- Equivalent to specifying the vols of the rates and a pseudo-root of the correlation matrix.

## abcd approach to vols

- We are given the implied vol of each caplet.
- We posit the existence of a function,  $\sigma(s)$ , the volatility of a forward rate with  $s$  years to go, such that

$$\int_0^{T_i} \sigma(T_i - s)^2 ds = \sigma_i^2 T_i.$$

- eg  $\sigma(s) = (a + bs)e^{-cs} + d$ ,
- and try to fit,  $a, b, c$  and  $d$  to the implied vols.
- Reduces  $n$  points to 4.
- Won't get perfect fit but do get a good fit.
- Integral of square is analytic.



# Interpretation of $a$ , $b$ , $c$ and $d$

- Volatility of a rate with long to go is  $d$ .
- So  $d$  also equals long term implied vols.
- $a + d$  is very short term volatility (both instantaneous and implied.)
- $b, c$  affect location of hump and rate of decay:

$$\frac{b - ca}{cb},$$

- We can use these facts to simplify the optimization.

# K factors

- Whilst we want realistic dynamics and time homogeneity, the most important thing is to price caplets precisely.
- We therefore solve for  $a$   $b$   $c$   $d$  and then put the vol of rate  $i$  to be

$$\sigma_i(t) = K_i((a + b(t_i - t))e^{-(t_i - t)} + d),$$

where  $K_i$  is precisely chosen to make the implied vol of the caplet correct.

# Formula for $K$ factors

- We have

$$K_i^2 = \frac{\sigma_i^2 T_i}{\int_0^{T_i} \sigma(T_i - s)^2 ds}.$$

- $K$ s should be close to 1.
- Difference from 1 expresses the failure of time homogeneity.

## Effects of curved volatilities

- Humped volatilities mean that information has different amount of effects on different rates.
- This causes extra decorrelation.
- The covariance between two rates is

$$\int_s^t \sigma_i(r) \sigma_j(r) \rho_{ij} dr,$$

- By Cauchy-Schwartz, this is less than

$$\hat{\sigma}_i \hat{\sigma}_j (t - s) \rho_{ij}.$$

## Effects of terminal decorrelation

- Humped vols lead to extra decorrelation known as *terminal decorrelation*.
- This can have marked effects on the price of exotics.
- How much of this occurs depends on the precise nature of the hump.
- This means that the exotic price is very sensitive to the two-year vols – traders tend to dislike this.

# Correlations

## Three main approaches

- Parametric, i.e. we choose a parametric form and try to get nice properties
- Historical, we use correlations of rates over say the last ten years
- Fitting, we try to get prices of instruments other than caplets correct

## Parametric approach

- What properties should correlation have?
- Time homogeneity
- Close together rates should be more correlated than far apart rates.
- Rates with a long time to go should have more correlation than rates with a short time to go

# Parametric forms

- Simplest:

$$e^{-\beta|t_i - t_j|}$$

- With  $\beta \sim 0.1$ .
- Correlation only depends on difference in maturity dates so time homogeneous.
- Very little correlation between two ends of the curve.
- Full factor



# Long Correlation

- We can eliminate problems with decay simply by adding in an extra term:

$$L + (1 - L)e^{-\beta|t_i - t_j|},$$

- We take

$$L \sim 0.5,$$

$$\beta \sim 0.2.$$

- In fact, we can do this to any parametric form without losing positive definiteness

## More sophisticated forms

- We should have that the 29-year rate is more correlated to the 30-year rate than the 2-year rate to the one-year rate.
- So make correlation a function of a power of time to go .
- E.g

$$\rho_{ij} = e^{-\beta||t_i-t|^{\gamma}-|t_j-t|^{\gamma}|}.$$

# Consequences of time-dependent correlation

- Get desired effect since

$$\sqrt{30} - \sqrt{29} < \sqrt{2} - \sqrt{1},$$

- But we now have to do the integral

$$\int_s^t \sigma_i(r) \sigma_j(r) \rho_{ij}(r) dr,$$

numerically.

- But only do it once per pair of rates per step so not too bad.
- Or just use mid-point approximation.

# Historical approach

- We take say ten years data.
- We measure the historical correlations between different parts of the curve over that period.
- These correlations have to be defined in terms of time to go, eg the rate from 1 to 2 years to go, from 2 to 3 years to go.
- Many noise and boot-strapping problems

## Historical approach using swap-rates

- Main market observable is swap-rates
- So can observe a historical correlation matrix for them.
- Swap-rates are co-initial
- Can map using  $Z$  matrix approach into forward-rate correlations.
- Still have to interpolate and applying the  $Z$  matrix is essentially differentiation, so this magnifies noise.

# Principal components

- Once we have this matrix, we perform a principal components analysis, and retain as many factors as we wish.
- i.e. we find the eigenvalues and eigenvectors, keep the largest  $F$  eigenvalues setting the others to zero.
- Rescale to keep everything as a correlation matrix.

# Reasonability

Any correlation matrix with reasonable properties yields:

- First principal component is roughly horizontal and is most of the variation
- Second principal component is a sloping line.
- Third principal component is a hump.
- Almost all variance eg  $> 95\%$  typically described by the first three components.

## Other instruments

- Main liquid vanilla derivative is the European swaption.
- We can therefore try to find correlations that cause us to match both caplet and swaption prices.
- To do this, effectively we need to know how to price swaptions quickly in the LMM.



## The need to calibrate to co-terminal swaptions

- Many instruments are callable.
- The decision to call will be heavily affected by the level of rates at exercise time.
- The swap-rate between the exercise time and the final maturity is therefore very important.
- We want to get its dynamics right (or least hedgable...)
- So price European option on that rate correctly.

## Mimicing swap-market models

- We have thus far only discussed the LMM.
- However, for calibrating to swaptions, swap-market models are much more effective (but worse for caplets...)
- One solution is to learn how to transform one into the other.
- We then fit each instrument where it is easiest to fit.

# Swaps in terms of forwards

- A swap-rate is determined by the forward rates underlying it.
- So if  $SR_{\alpha,\beta}$  runs from  $t_\alpha$  to  $t_\beta$  it is a function of

$$f_\alpha, f_{\alpha+1}, \dots, f_{\beta-1}.$$

- We can therefore deduce its dynamics from those of forwards.

# Jacobian equation

- Rates are lognormal so it is the relationship between the logs that matter.
- Ignoring drifts

$$d \log \text{SR}_{\alpha, \beta} = \sum_j \frac{\partial \log \text{SR}_{\alpha, \beta}}{\partial \log f_j} d \log f_j.$$

- The Jacobian will be state dependent but we can pretend that it isn't.

# Z-matrix

- If we pick  $n$  swap rates, eg co-terminal that is with same termination date, then the Jacobian,  $Z$ , will be invertible:

$$Z_{ij} = \frac{\partial \log \text{SR}_i}{\partial \log f_j} = \frac{f_j}{\text{SR}_i} \frac{\partial \text{SR}_i}{\partial f_j}.$$

- Up to drifts (which are determined by no arbitrage):

$$d \log \text{SR} = Z d \log f.$$

# Invertibility of $Z$

- If we know the co-terminal swap rates, we know the bond ratios.
- If we know the bond ratios, we know the forwards.
- So the map between log forward rates and log swap rates is invertible.
- Its derivative,  $Z$ , is invertible as well.

## Effective use of the $Z$ -matrix

- We use  $Z(0)$  as a substitute for  $Z$ .
- If  $A$  is the pseudo-root for  $f$  across a step then  $ZA$  is the pseudo-root for  $SR$ .
- Note that the number of factors doesn't change.
- Clearly, can go opposite way.
- Allows one to map between FRA land and swap-land.

# Truncation

- One has to be slightly careful.
- The forwards underlying a swap keep moving even after it has reset.
- So one must make sure that the swap stops at the right time.
- In practical terms, we must zero the relevant rows of the swap-pseudo root once they have stopped.



## Calibrating LMM to coterminal swaptions

- Getting the co-terminal swaptions correct in co-terminal swap MM is easy.
- Simply ensure that the total volatility for each swaption is correct.
- So we can calibrate in swap-land and then map back to FRA-land.
- Thus getting the coterminal swaptions right in the LMM.

## One approach

- Calibrate LMM to caplets, using a historical correlation matrix, and time homogeneity
- Map to the equivalent volatility structure in swap-land using the  $Z$ -matrix
- Rescale co-terminal swap vols to get co-terminal swaptions correct (i.e. multiply pseudo-root by a diagonal matrix with entries the ratio of market vol to calibrated vol)
- Map back to FRA land

## Time homogeneity in co-terminal land

- In LMM, all rates have the same tenor and thus are qualitatively the same.
- In co-terminal swap MM, this is not true.
- However, for each tenor we can observe the implied swaption vol for every maturity, and then fit an a b c d curve to infer a time homogeneous evolution.
- Add in historical correlation, and we are done. (map back to LMM at end)

## Features of this approach

- Co-terminal swaptions are priced correctly.
- Evolution is roughly time homogeneous.
- FRA correlations are an input.
- Caplets are NOT priced correctly

## Alternative approaches

Our approaches so far have the following features

- Time homogeneous
- Use of historical correlations
- Fit only  $n$  instruments
- Emphasis on realism of dynamics.

Some people prefer to fit as many instruments as possible and forget about realism - philosophy if something is hedgeable then realism does not matter.

## Do a global fit

- Pick a set of instruments to calibrate to
- Pick a number of factors.
- Do a least-squares global fit on the pseudo-root elements, (using swaption approximation formulas) (or parametrize and fit over the parameters.)
- Is the entire swaption matrix truly consistent?

## Pros and cons of global fits

- Get more instruments approximately correct.
- No realism.
- Price has a sensitivity for better or worse to every instrument fitted to.
- Instability.

## Direct swaption approximation formula

- Using the Z-matrix, we can deduce a direct formula for implied volatility of a swaption from FRA covariances.
- Let SR set at time  $T$ ,
- Let  $C_{jk}$  be the covariance of log forwards  $j$  and  $k$  underlying SR,
- We have the approximation

$$\sigma_{\text{SR}}^2 T \sim \sum_{j,k} \frac{\partial \log \text{SR}}{\partial \log f_j}(0) \frac{\partial \log \text{SR}}{\partial \log f_k}(0) C_{jk}.$$



# Hull–White approximation

- If use this approach to approximate the variance of a swaption, we can get a formula for its implied vol.
- This is the Hull-White approximation:

$$\sigma_i^{\hat{S}^R} = \sqrt{\frac{1}{T} \sum_{j,k \geq i} Z_{ij} Z_{ik} C_{jk}}.$$

## Reduced-factor models

- It's normal to work within a reduced factor LMM.  
(Computational time is proportional to factors)
- We can achieve this either by calibrating within such a model from the start.
- Or by reducing the number of factors at the end.

# How to factor reduce

- Decompose covariance matrix for each step into product of correlation matrix and standard deviations.
- Compute eigenvalues and eigenvectors of correlation matrix.
- Put  $n - F$  smallest eigenvalues to zero.
- Rescale to make modified correlation matrix have 1s on the diagonal.
- Multiply back up by standard deviations.

## Consequences of factor reduction

- We preserve caplet implied vols.
- We increase correlation between forwards.
- We will change the swaption implied vols.
- If we work in swap-land then we can preserve swaption implied vols, but will mess up caplet implied vols.
- Can easily only fix  $n$  instruments as usual

# Factor reduction and step-size

- It's important to realize that factor reduction and discretization do not commute.
- If particular, suppose we have an  $F$ -factor model, with  $A_1$  and  $A_2$  the first two pseudo-roots.
- Then the covariance matrix for the first two steps will be

$$C = A_1 A_1^t + A_2 A_2^t,$$

- $C$  will generally be of order  $2F$  not  $F$ .

## Personal viewpoint

- Don't factor reduce.
- Instead, calibrate within a reduced factor model from the start.

## Simultaneous Calibration to caplets and swaptions

- We want to calibrate the LMM to both caplets and co-terminal swaptions.
- Realistic correlations, plus time homogeneous volatilities for a fit to one, does not give a fit to the other.
- So we must fit both at once.

## Work in co-terminal land

- Our approach is to find a calibration of the co-terminal SMM which matches both sets of prices,
- Then map back via  $Z$  matrix to a calibration of the LMM.



# Caplets as swaptions

- The fundamental observation of this approach due to GHLS is that a forward can be viewed as a linear combination of two neighbouring swap-rates.
- We can write

$$f_i = w_{i,0} SR_i + w_{i,1} SR_{i+1} .$$

- The second term will be large negative (eg -10) and the first large positive (eg 11).

# Hull and White formula

- There are many ways to compute the coefficients of the approximation.
- We use a modified Hull–White formula.
- The  $Z$  matrix approach gives us the Hull–White approximation for the variance of log–forward  $j$

$$\sum_{k,l \geq j} Z(0)_{jk}^{-1} \tilde{C}_{kl}^{\text{SR}} Z(0)_{jl}^{-1}.$$

# Chopping Hull and White

- GHLS suggest truncating. i.e. drop all terms with  $k$  or  $l$  bigger than  $j + 1$ .
- These are about 0.1% of the size of the first two terms but there are  $(n - j)^2$  of them. We therefore set

$$w_{j,0} = Z(0)_{jj}^{-1} + \sum_{k=j+2}^n Z(0)_{jk}^{-1},$$

$$w_{j,1} = Z(0)_{j,j+1}^{-1}.$$

# Our approximation

- We then get our approximation for the caplet variance to be

$$\tilde{\sigma}_j^2 t_j = w_{j,0}^2 \tilde{C}_{jj}^{\text{SR}} + 2w_{j,0}w_{j,1} \tilde{C}_{j,j+1}^{\text{SR}} + w_{j,1}^2 \tilde{C}_{j+1,j+1}^{\text{SR}}.$$

# Factors and correlations

- We want to calibrate to  $2n - 1$  instruments.
- In an  $F$ -factor model, we have  $nF$  pseudo-root elements per step.
- So we have much freedom.
- We therefore assume that a correlation pseudo-root is given for each step.
- Our objective is then to determine the vol of each swap-rate for each step.
- Still  $n(n - 1)/2$  elements to determine.

## Time homogeneity

- If we drop the requirement to calibrate to caplets, then we can find a time homogeneous fit to the swaption matrix.
- This is chosen so that the implied swaption matrix in the future is the approximately the same as today's.
- This gives us a swap-rate vol for each time step.
- But does **not** match the caplets.

## Stepping through

- We have reduced the caplet implied variance to a quadratic in two swap vols.
- We can step through the rates one by one matching one caplet vol at a time and keeping the swaption vol correct.
- For the first swap-rate, we have only one step and we use the fitted vol.

# Bootstrapping

- Now suppose, we have determined the vol for each step of  $SR_j$ .
- We want to choose the vol of  $SR_{j+1}$  so that the caplet of  $f_j$  is correct and the implied vol of the swaption on  $SR_{j+1}$  is correct.
- We can choose the vol of  $SR_{j+1}$  for each time step.
- So we have a  $j + 1$  vector to work with.



# Bootstrapping II

- We have 2 constraints: the caplet and swaption.
- Let the vector be  $z$ : these can be written as

$$\sum_{i \leq j} z_i^2 = R^2,$$
$$A + \sum_{i < j} B_i z_i + \sum_{i < j} z_i^2 = S^2.$$

# Interpreting the constraints

- Constraint I says that  $Z$  lies on a sphere.
- Constraint II says that  $Z$  lies on a cylinder centred at  $-B/2$ .
- Generally can expect a  $j - 2$  dimensional sub-manifold of solutions.
- We want to find the point on this sub-manifold closest to the time-homogeneous solution.

## Using geometry

- We can simplify the problem.
- The sub-manifold is invariant under rotations of the first  $j - 1$  coordinates fixing  $O$  and  $-B/2$ .
- This means that we can reduce to a 3-d problem.
- In particular, we rotate  $-B/2$  to  $(\alpha, 0, 0, \dots, 0)$ .
- And  $X$ , the homogeneous solution, to  $(x_1, x_2, 0, \dots, 0, x_j)$ .

# The 3-d problem.

- The swaption equation gives  $z_j$  from  $z_1$  and  $z_2$ .
- The caplet equation gives  $z_1$  from  $z_2$  :

$$z_1 = \sqrt{S_j^2 - (z_2 - \alpha)^2},$$

$$z_j = \sqrt{R_j^2 - z_1^2 - z_2^2}.$$

# Minimizimizing

- We then do a search on  $z_2$  on the interval that makes these two equations have valid solutions.
- To minimize the distance to the rotated  $X$ , we use a sum of squares metric, and use Brent's method.
- Since this is one-d and convex, it's benign.

## Rotating back

- Once we have the solution, we rotate back.
- We have constructed the solution closest to time homogeneous which matches both vols.

# Constructing the rotation

- A rotation is simply a choice of ortho-normal basis for  $R^j$ .
- So we construct a basis, by taking  $-B/2$  and  $X$ , and then adding  $e_i$  until we have a complete basis.
- We then perform Gram-Schmidt.

## Final steps

- We loop through the rates.
- We now have a co-terminal model with swaptions correct and caplets correct up to the accuracy of chopped HW.
- We map to FRA land i.e. the LMM with the  $Z$  matrix.
- We have caplets correct up to a small error due to using two different approximations: full HW and chopped HW. E.g. 0.1% error
- Swaptions are correct up to accuracy of full HW



# Iteration

- Standard trick: if the output is too big by a factor  $W$ , divide the input by  $W$ .
- So we divide each input caplet implied vols by the ratio of output to input.
- And then recalibrate.
- This gives 1e-6 accuracy after one iteration.

# Summary

- Caplet implied vols correct to  $1e-6$  (or more.)
- Swaption implied vols correct to accuracy of HW.
- Solution maximally time homogeneous.
- Correlations exogeneously specified.
- Solving is only 1d and so stable.
- Works for the reduced-factor displaced diffusion LMM by construction.

# Multi-periods

- In some markets caplets and swaps have different payment bases.
- E.g. in Euro, swaps are 1 year and caplets are six months.
- A callable product with coupons based on six month rates will need to be calibrated to both.
- We have a period mismatch.

# $Y$ Matrix

- We can infer dynamics for 1 year rates from those of six month rates
- By computing the log Jacobian,  $Y$ , just as for swaps and forwards.
- Won't be invertible as not square.
- However, it allows us to rapidly price 1 Year swaptions in the six month model.

## Calibrating to 1-year swaptions

- Pretend 1-year swaptions are six month swaptions.
- Calibrate six month model.
- Compute implied 1-year swaption vols.
- Rescale input volatilities by error.
- Repeat until error small. (i.e. once or twice.)

# References

- F, Ametrano, M. Joshi, Smooth simultaneous calibration of the LMM to caplets and coterminal swaptions, SSRN
- S. Galluccio, JM Ly, Z. Huang, O. Scaillet, Theory and calibration of swap market models, Math. Finance Vol 17/1, Jan 2007
- P. Jackel, R. Rebonato, Linking caplet and swaption volatilities in a LIBOR market model setting, J. Computational Finance, 6, 35–45
- M.S. Joshi, the Concepts and Practice of Mathematical Finance, Dec 2003
- M.S. Joshi, J. Theis, Bounding Bermudan swaptions in a swap-rate market model, Quantitative Finance 2 No 5 (October 2002) 370–377
- R. Rebonato, Modern Pricing of interest rate derivatives, 2003

## Part VI

### Skew in the LMM

## Why the LMM is not enough

- Objective: *Price exotic interest-rate derivatives with a model that prices all caplets correctly and implies believable future dynamics for caplet prices.*
- LMM only fulfills this when there are no smiles.



# Market smiles

Two main components

- Skew, downwards sloping
- Implied vols increase for large strikes

## Observed dynamics

- Implied vols and historical vols are much higher when rates are low.
- This suggests that rates are not log-normally distributed.
- General belief is that they are somewhere between normal and log-normal.

# CEV

- One natural way to obtain skew is CEV:

$$df = \sigma f^\gamma dW_t,$$

- Conceptually appealing but difficult to compute with.
- One can view CEV as a geometric interpolation of normal and log-normal.
- It can be done, however, Anderson/Andreason.

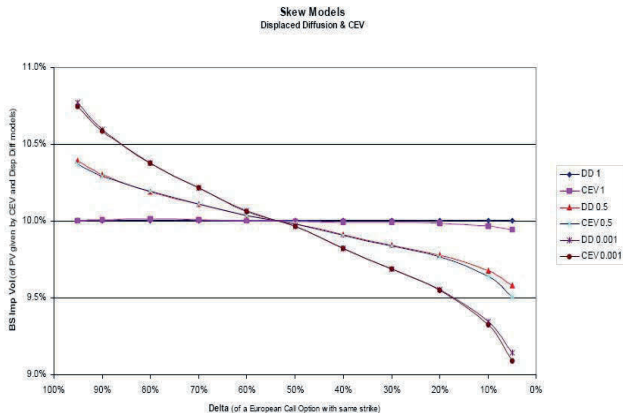
# Displaced diffusion

- Take arithmetic mean of log-normal instead:

$$\frac{d(f + \alpha)}{f + \alpha} = \sigma dW_t.$$

- i.e. make  $f + \alpha$  lognormal instead of  $f$ .
- Marris showed that displaced diffusion and CEV gave very similar smiles and dynamics.

# CEV vs DD



## Displaced diffusion as local vol

- If we want to understand relative to lognormal, we can write:

$$\frac{df}{f} = \sigma \frac{f + \alpha}{f} dW_t.$$

- This will clearly increase as  $f$  gets small.

## Pricing caplets with displaced diffusion

- We have

$$(f - K) = (f + \alpha) - (K + \alpha).$$

- We can regard a caplet as a call option on a log-normal rate struck at  $K + \alpha$ .
- Simple recipe: just add  $\alpha$  to forward and strike in the Black formula.
- Also works for implied vols.

## Negative rates

- If we make  $f + \alpha$  lognormal, then its smallest value is  $-\alpha$ .
- This means that we can have negative rates. Typically, we have around 2% displacement.
- Not a huge problem but that depends on product.
- E.g. zero strike digital caplet...
- More of an issue when initial rates are low.



## Displaced diffusion vs CEV

- DD - easy caplet formula
- DD - positive probability of negative forward rates
- DD - easy simulation
- CEV - positive probability of zero forwards
- CEV - difficult to simulate
- Both converge to normal in one extreme and lognormal in the other

# Ideal model

The ideal model would have

- Zero probability of rates less than or equal to zero
- Implied vols increasing as strike decreases in a bounded way
- Analytic formulas for caplet prices
- Easy to simulate

Not yet invented...

# Parameter limits

- We have

$$\frac{P_{j+1}}{P_j} = \frac{1}{1 + f_j \tau}.$$

- So we must have

$$1 + f_j \tau_j > 0, \implies \alpha_j < \frac{1}{\tau_j}.$$

- If we take  $\alpha_j = \frac{1}{\tau_j}$ , bond ratios are log-normal. “String model” or essentially normal rates.

# Vol equivalence

- ATM Black formula approximation

$$\text{Black} \sim Cf\sigma_{LN}\sqrt{T_\tau}P_{T+\tau}.$$

- ATM Black formula approximation in displaced land

$$\text{Black} \sim C(f + \alpha)\sigma_D\sqrt{T_\tau}P_{T+\tau}.$$

- For these to give the same price

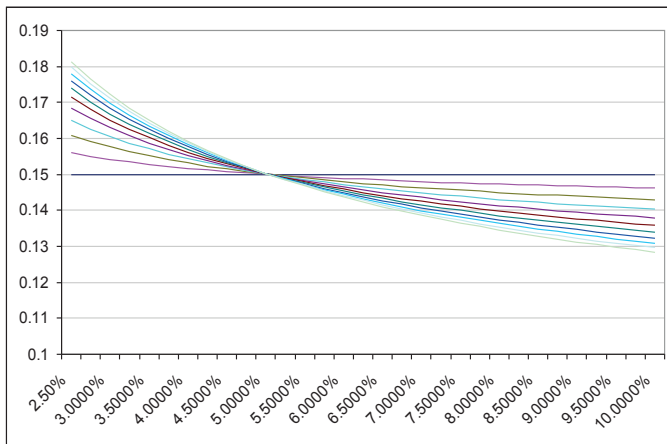
$$\sigma_D(f + \alpha) = \sigma_{LN}f,$$

- So

$$\sigma_D = \frac{f}{f + \alpha}\sigma_{LN}.$$

- Or just use the implied volatility function.

## Displaced diffusion skews



# Adapting LMM to displaced diffusion

- Each rate  $f_j$  follows a process

$$\frac{df_j}{f_j + \alpha_j} = \mu_j dt + \sigma_j dW_t,$$

with the Brownian motions correlated.

- The model makes sense mathematically with a different displacement for each  $j$ .
- But time inhomogeneous.

# Drifts in DD LMM

- We saw before that in the terminal measure for any vol function:

$$\mu_j = - \sum_{k=j+1}^{N-1} \frac{\tau_k \sigma_k(f_k, t) \sigma_j(f_j, t) \rho_{jk}}{1 + f_k \tau_k}.$$

- So we set in the DD LMM:

$$\sigma_k(f_k, t) = (f_k + \alpha_k) \sigma_k,$$

- and we have

$$\frac{\mu_j}{f_j + \alpha_j} = - \sum_{k=j+1}^{N-1} \frac{(f_k + \alpha_k) \tau_k \sigma_k(t) \sigma_j(t) \rho_{jk}}{1 + f_k \tau_k}.$$

## Drifts in DD LMM 2

- The terms in the drift involving volatility must be adjusted to take account of displacement, those involving discounting do not change.
- Similarly in swap-rate market models.
- The fast algorithm applies as before



## Drifts in DD LMM 3

- Similarly, in spot measure for DD LMM, we have

$$\tilde{\mu}_j = \frac{\mu_j}{f_j + \alpha_j} = \sum_{k=0}^j \frac{(f_k + \alpha_k) \tau_k \sigma_k(t) \sigma_j(t) \rho_{jk}}{1 + f_k \tau_k}.$$

- So

$$d \log(f_j + \alpha_j) = \left( -\frac{1}{2} \sigma_j^2(t) + \tilde{\mu}_j \right) dt + \sigma_j(t) dW_t.$$

## Evolving DD LMM

- Procedure essentially as before.
- We work with  $\log(f_j + \alpha_j)$  rather than  $\log(f_j)$  in our code.
- PC and IPC go over with no real changes except the new drift computation.
- Glasserman–Zhao can also be made to work (see Brace.)

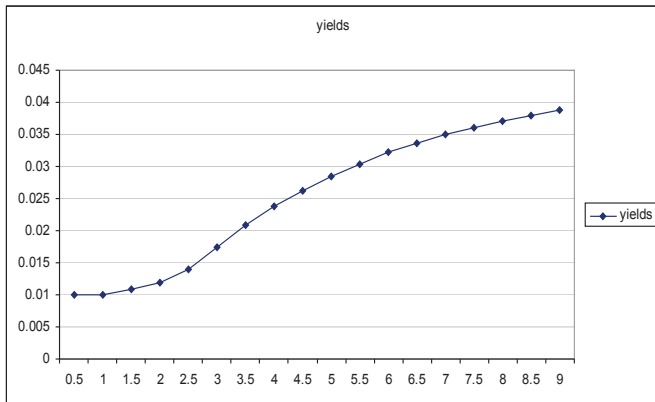
## Calibrating DD

- We can apply the same techniques as before.
- Main issues are in figuring out where to put in the displacement.
- General approach:
  - Pick displacement to match market skew
  - Convert all volatilities into volatilities for the displaced model
  - Work with displaced implied volatilities
  - Work with vol curves for displaced implied vols
  - And displaced swaption implied volatilities.

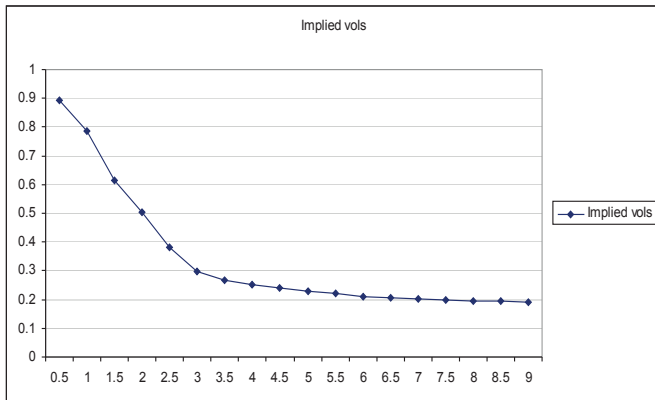
## Curve shape and time homogeneity

- Vols tend to go up when rates go down.
- So if the curve is steeply increasing eg USD curve 5 years ago then short term vols will be very high and long term vols less so.
- This can make a b c d very hard to fit for log-normal model.
- But displaced implied vols are more benign.

## Increasing yield curve



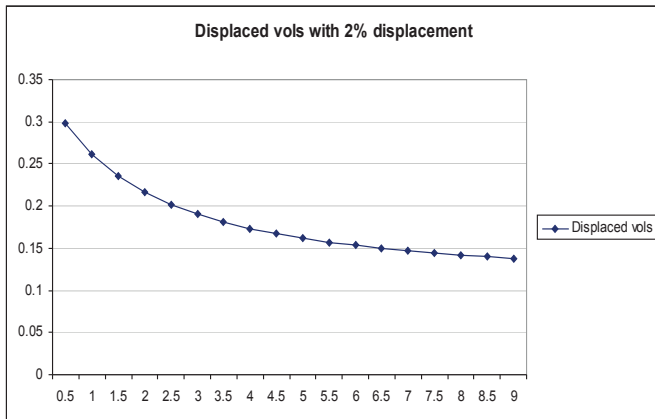
## LN Implied vols inc yield curve



## Displaced vols discussion

- The displaced vols display a much more sensible shape.
- Reflects market belief that interest rates are somewhere between normal and log-normal.

## DD Implied vols inc yield curve





# Z matrix

- When working with the undisplaced model we used the log Jacobian to map between the forward and swap cases.
- We can do the same here:

$$Z_{ij} = \frac{\partial \log(SR_i + \alpha)}{\partial \log(f_j + \alpha)}.$$

# Simplifying the Z-matrix

- We can write

$$Z_{ij} = \frac{f_j + \alpha}{SR_i + \alpha} \frac{\partial SR_i}{\partial f_j}.$$

- So if we have the undisplaced matrix, getting the displaced one is easy!
- Rest of calibration is the same as in undisplaced case.

## Relating displacements

- Generally, I recommend using one displacement for all forwards and co-terminal swap-rates.
- Otherwise, one gets scrambling as one maps between FRA and swap land.
- But maths does work if you really want to!

## Where does displaced diffusion make the biggest difference?

- Products with discontinuities.
- The price of a digital option is highly skew dependent.
- In any model, a digital caplet is worth:

### **Black Lognormal Digital Price - Skew times Vega**

- So if your product has big jumps, get the skew right eg TARN.
- (“concepts” chapter 7.)

## Skew from displacement

- We can invert the relation on the previous slide.

- 

$$\text{Skew} = \frac{\text{LNDC} - \text{DDDC}}{\text{Vega}},$$

- LNDC = log-normal digital caplet price,
- DDDC = displaced diffusion digital caplet price,
- Vega = vol derivative of (log-normal) caplet with same strike ,
- all these using the implied vol to get the caplet struck at K have the correct price.

## Bottom-line

- The extra effort in making a log-normal implementation also a displaced-diffusion one is tiny.
- It buys you skew and often easier calibration at very little computational cost.
- You can always set the displacements to zero.
- So include it from the start!

## Part VII

Lower Bounds for early exercisable products

## Early exercise

Many interest rate derivatives involve the ability to “call” or early terminate. For example, the Bermudan swaption:

- The right to enter into a swap at a fixed strike
- Exercisable on any one of a number of dates
- Typically the swap always has the same end-date.
- Also arises as the right to break a swap.



# Callable Bonds

Many contracts are structured as a bond paying a fancy coupon that can be called.

- Investor pays bank principal upfront.
- Bank pays investor coupon that is a function of interest rates, e.g.  $\max(K - 2L, 0)$  times accrual every six months with  $L$  six month LIBOR
- On any coupon date, bank can terminate the deal by repaying the principal.

## Callable bonds versus swaps

- The principal can be invested to pay a floating rate so the callable bond is typically regarded as a breakable swap between a coupon paying complicated function of interest rates and a floating rate.
- So pricing cancellable swaps between complicated coupons and floating rates is generally sufficient.

## Bermudan optionality

- A Bermudan option is an option that be exercised on one of a fixed finite number of dates.
- It is typically Bermudan options that arise with interest rate derivatives and particularly with the LMM.

## Optimal exercise strategies

- For a Bermudan option, we should exercise if and only if the continuation (i.e. non-exercise) value is less than the exercise value.
- So an optimal exercise strategy exists and is easily described.
- But requires knowledge of the continuation value which may not be available.

# Why Monte Carlo?

- Lattice methods are natural for early exercise problems, we work backwards so the continuation value is always known.
- Lattice methods work well for low-dimensional problems but badly for high-dimensional ones.
- Path-dependence is natural for Monte Carlo
- LIBOR market model difficult on lattices

# LMM and lattices

- Lattices require a low-dimensional state space.
- If we use  $F$  random numbers per step, we have a  $kF$  dimensional state space after  $k$  steps
- This arises from the state dependence of drifts and the time dependence of vols.
- It is possible to approximate with models that surmount these problems but the approximations are not great.
- Plus path-dependent callables e.g. snowball

## Naive approach

- At each exercise time, we run an inner simulation to estimate the continuation value.
- The problem is that when we do the inner simulation, we then need to run a sub-sub simulation at the next exercise date and so on.
- Clearly, not feasible.

# Buyer's price

- Holder can choose when to exercise.
- Can only use information that has already arrived.
- Exercise therefore occurs at a stopping time.
- If  $D$  is the derivative and  $N$  is the numeraire, the value is therefore

$$\frac{D_0}{N_0} = \sup_{\tau} \mathbb{E}(N_{\tau}^{-1} Z_{\tau}).$$

- Expectation taken in martingale measure.
- The pay-off at time  $t$  is  $Z_t$ .



## Justifying buyer's price

- Buyer chooses stopping time.
- Once a stopping time has been chosen the derivative is effectively an ordinary path-dependent derivative for the buyer.
- In a complete market, the buyer can dynamically replicate this value.
- The buyer will maximize this value.
- Optimal strategy: exercise when **continuation value less than or equal to exercise value**.

# Lower bounds

- The buyer's price is the supremum over all exercise strategies.
- So any choice of an exercise strategy will give us a lower bound.
- Many methods of finding such strategies now exist.

## Two passes

- Most methodologies for doing early exercise are two pass approaches:
  - 1 First pass: estimate the optimal exercise strategy, generally working backwards on a model set of paths.
  - 2 Second pass: use the estimated strategy to get an unbiased estimate of a lower bound on the option price, by using an independent (i.e. different seed) second set of paths
- Single pass methods generally give an **upwards**-biased estimate of a *lower* bound

## Model paths

- Typically, we generate a fixed set of model paths e.g. 16384.
- We work backwards with deflated values throughout.
- The continuation value at a given point on a given path is then taken to be the value at the time of exercise in the future on the *same* path.
- This can be regarded as a very noisy 1-path MC estimate.

## Noise removal

- The noise is reduced by aggregating information from neighbouring paths.
- Various algorithms differ in
  - 1 How information aggregation is done
  - 2 How exercise is decided.
- Two main approaches:
  - 1 Maximize value via optimization using a parametric exercise strategy.
  - 2 Regression to estimate continuation value and hence strategy.

## Carriere/Longstaff-Schwartz

- The most popular method currently for doing early exercise with Monte Carlo.
- This uses least-squares regression to estimate the continuation value.
- The crucial issue is that one has to choose basis functions to regress against.

# Least-squares regression

- We have a collection of pairs  $(x, y) \in R^n \times R$
- We have a collection of basis functions  $\psi_j(x)$ .
- We want to find the coefficients  $\alpha_j$  which minimize

$$\sum \left( y_j - \sum \alpha_j \psi_j(x) \right)^2.$$

- Differentiating with respect  $\alpha$  yields

$$\sum_i 2 \left( y(x_i) - \sum_j \alpha_j \psi_j(x_i) \right) \psi_r(x_i) = 0$$

- We can write this as

$$C\alpha = D$$

where  $C_{jr} = \mathbb{E}(\psi_j \psi_r)$  and  $D_j = \mathbb{E}(\psi_j y)$ .

## Least squares regression again

- This means we have to solve the matrix equation

$$C\alpha = D.$$

- But  $C$  will be a small matrix and generally non-singular unless one basis function is a linear multiple of the others on the sample set.
- This is easy.



## Alternate methodologies

- There is nothing forcing us to use least-squares regression.
- We can use alternative regression methodologies.
- Least-squares regression is highly basis function dependent - suggests non-parametric methodologies.

# The Longstaff-Schwartz algorithm

- Generate a set of model paths
- Work backwards.
- At final time, exercise strategy and value is clear.
- At second final time, define continuation value to be the deflated value on the same path at final time.
- Regress continuation value against a basis.
- Use regressed value to decide exercise strategy.
- Define value at second last time according to this strategy and value at following time.
- Work backwards.

## Definition of continuation value

- In LS, we have two ways to get the continuation value
  - 1 Use the continuation value predicted by the regression
  - 2 Use value at next time frame on same path
- LS recommend method 2.
- Less to go wrong.

## Choice of basis functions

- How do we choose the basis functions?
- Typically want them to mimic the shape of the pay-off.
- E.g. for a Bermudan swaption, use a small number of European swaptions. (requires swap-rate market model.)
- Brace says *More art than science*.
- Discount factors, exercise values, swap rates are all simple choices that tend to do OK but not brilliantly.

## Improving Longstaff–Schwartz

- We need an approximation to the unexercise value at points where we might exercise.
- By restricting the domain, approximation becomes easier.
- Exclude points where exercise value is zero.
- Exclude points where exercise value less than maximal European value if evaluable.

## Case study: Bermudan swaption

- Possible choices of basis functions, Longstaff-Schwartz:
- Use current Swap rate
- Its square
- The discount factors

## Further choices

- Forwards: Forward rates
- Simpler : use first three powers of next rate, and its annuity
- Swaps: use co-terminal swap rates
- Swaps values: use values of swaps
- Swap annuities: use annuities of swaps
- Swap values and forwards
- Coterminal swap forward: use next forward and contiguous swap rate and take exponentially weighted polynomials

# Swaptions mid

- Use next European swaption, last European swaption and one in the middle.
- Only 3 basis functions.
- Requires Swap-market model.



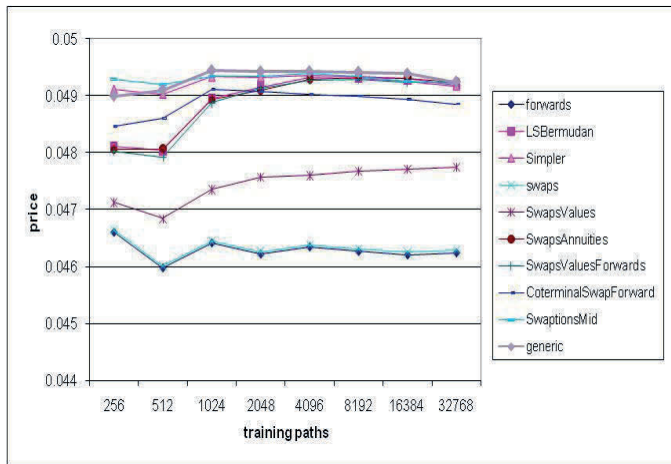
## Generic basis functions

- First forward rate that hasn't reset.
- Swap-rate to end which is contiguous but not overlapping to it.
- Final discount bond
- Quadratic polynomials in them
- The value of the floating leg to the end is therefore within the basis

## 10NC1 Bermudan full factor

- Convergence with training paths
- Roughly at the money
- 65536 Sobol paths for second pass
- Second pass converged to less than 1 bp
- Mersenne for first pass

# Convergence for various bases



# Typical weights for swaptions mid

| first     | middle   | last      | sum      |
|-----------|----------|-----------|----------|
| -0.324892 | 1.417415 | -0.066879 | 1.025644 |
| -0.139138 | 1.066469 | 0.150421  | 1.077752 |
| -0.056438 | 0.996663 | 0.124341  | 1.064567 |
| -0.026319 | 0.974162 | 0.105046  | 1.05289  |
| -0.019848 | 1.010127 | 0.036481  | 1.02676  |
| -0.020091 | 1.027272 | 0         | 1.007181 |
| -0.014571 | 1.019698 | 0         | 1.005126 |
| 0.010059  | 0.979982 | 0         | 0.990041 |

## Adaptive basis functions

- Doing a search over lots of basis functions, every time we change the pay-off is time-consuming and annoying.
- Ideally, we want a set of basis functions that always works.
- This seems too hard.
- Instead, develop a methodology for automatic basis function selection.

## Automatic extension

- Pick a core set of variables. Eg a few rates
- Pick an extended set of variables. Eg some bonds
- For each element in the extended set, add it to the core set and take all quadratic polynomials in the new set.
- This gives us a set of basis of functions for each element of the extended set.

## Comparison

- At each exercise date, for each possible extended set and the original set, we perform a regression.
- We select the one with the best modified  $R^2$  statistic.
- i.e. the best fit whilst penalizing for adding in the extra functions when comparing with the original set.

## Generic choice

- For the core set of variables, use the first forward rate, and the co-terminal swap-rate not overlapping it.
- If path-dependent, use a variable such as the current coupon to capture it.
- For the extended set, use ZCBs. If there are a large number of dates use a half or third of the ZCBs.



# Breakables

- LS and similar methods depend on the idea that the derivative is exercised once and then one receives a cash sum.
- Whereas most IRD products work on receiving a stream of cash-flows until breakage occurs.
- Right to break can be viewed as right to receive value of the negative of the opposite product. This is typically done

## Example: callable inverse floater

- The opposite value on breakage is a sum of caplet prices.  
Present value of

$$\sum \max(K - 2f_i, 0)\tau_i P_{i+1}.$$

- These values are highly model- and calibration-dependent.
- Can be done but only with difficulty.
- If coupons are fully path-dependent then forget it.

## Working with the breakable directly

- Instead of defining the continuation value to be the cash sum at exercise, we take the deflated value of the cash-flows along the path until the time of breakage.
- The LS algorithm then works without problems.
- It is then totally model independent.

## Excluding sub-optimal exercise

- If we are on a break date, there are certain points where we would never exercise.
- Suppose the PV of the cash-flows between now and next exercise date is positive, then do not break.
- We can therefore restrict domain for regressions.

## Alternative approach

- Regard coupons as arriving at the time of breakage.
- So any coupon that arrives beforehand is held in a bank account and rolls up until the product terminates.
- Pay-off is now equivalent to a single payment and LS applies directly.
- Note that no longer truly an option but this doesn't matter.
- (Bender-Kolodko-Schoenmakers 2006)

## Alternative approach II

- Piterbarg. Use two regressions:
  - One for the continuation value
  - One for the exercise value

Features:

- extra computational work,
- extra noise
- Doesn't work for upper bounds.
- Does apply when the choice is between two complicated streams rather than a stream and nothing.

## The parametric approach (Anderson)

- Suppose we know (or guess!) that the exercise boundary takes some particular form.
- E.g. for a Bermudan swaption at step  $i$  :

$$SR_i > H_i$$

- We can optimize over  $H_i$  to maximize the price.
- Naive approach: global optimization.

# Backwards induction

- Similarly to LS we can work backwards.
- At second last time, we find the value of  $H_{n-1}$  that maximizes the value.
- We then define our exercise strategy using this.
- Continuation value defined by 1 pt MC as usual.
- Work backwards.
- Many 1d optimizations instead of 1 global optimization



# Optimization with parametric forms

- Suppose we want to do a 1d optimization over  $H_i$ .
- We therefore compute the sensitivity by a small shift and reprice.
- We get zero change!
- The result of an MC simulation is a piecewise constant function of  $H_i$ .
- A path has to cross the barrier to show up.
- Use non-derivative methods or large bumps.

## Choosing a parametric form

- The main problem is finding a class of forms that accurately describe the boundary.
- Boundary will be  $n - 1$  dimensional in  $n$  dimensional space with  $n$  large e.g. 30.
- One approach is to use a non-recombining tree and then project onto various two dimensional planes until one finds one that does a good job of separating exercise from non-exercise

## Bermudan swaption again

- After studying trees, Jäckel suggests using

$$f_i - \left[ \frac{p_1}{SR_{i+1} + p_2} + p_3 \right]$$

- Note that the choice of forward rate and swap-rate do not overlap.
- Approach is labour intensive.

# From Jäckel's paper

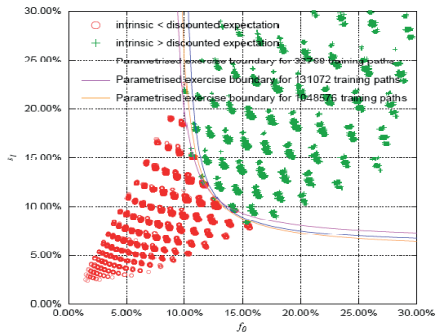


Figure 6: The exercise domain in the  $f_t - S_{t+1}$  projection of the evolved yield curve at  $t_i = 2$ , together with the parametrised exercise boundary resulting from training with different sizes of the training set.

## Merged approach: LSA

- At each exercise date, first do a least-squares fit to the continuation value:  $f$
- Then add a parameter,  $c$ , to  $f$  to get a parametrized set of estimates of continuation values.
- Optimize over  $c$  to get the highest price.

## LSA II

- This is effective.
- Remember all that matters is getting the boundary correct.
- Having a great estimate away from the boundary is not useful.
- LSA saves us from guessing the parametric form whilst optimizing to maximize value

# LSA+

- LSA is effective but often implies exercise at points that are provably sub-optimal.
- So we can combine this with exclusion of provably sub-optimal points to get a really good strategy.

## Double regression

- A related idea is double regression.
- When we do a global least-squares, we are fitting over points far from the boundary where the decision is clear.
- So once you have identified which points are close to the boundary, perform a second regression over just them, and use that as the continuation estimate close to the boundary.



## Double regression II

- Closeness to the boundary is defined as the absolute value of the difference between exercise and continuation values being less than a cut-off.
- The main issue is where to put the cut-off.
- We found 0.03 times the notional was effective.

# Delta-hedging

- When running the second pass, we want to reduce variance.
- In theory, if we delta hedge continuously using the ZCBs for reset dates then we can achieve zero variance.
- So let's hedge within in our pricing simulation.

## Delta hedging II

- We need to know the Deltas.
- But we already have an estimate of the continuation value from the LS method so we simply differentiate it – this is easy for reasonable basis functions.
- We hedge at each time that we would have evolved to originally.
- Inserting extra steps is not worthwhile, since Deltas are only approximate.

## Delta hedging and regression

- The needs for the Deltas are different than for the exercise strategy.
- We want a good Delta everywhere not just near the boundary and want a good approximation out of the money.
- So can do separate regressions with different emphases.
- Regression is fast so this doesn't slow us down.

# Smearing out

- We don't get an estimate of the basis functions at time zero with the normal LS method.
- We therefore have to start with some random yield curves to get a regression at time zero.
- Some people recommend doing this for the exercise strategy always.

## Using sub-simulations: policy iteration

- Sometimes standard methods only give us an OK exercise strategy.
- Everything is just too complicated.
- One solution is then to use sub-simulations to estimate the continuation value, but using the OK exercise strategy within the sub-simulation.
- Kolodko-Schoenmakers 2006

## Proving PI works

- It's only guaranteed to work if there's no Monte Carlo error.
- At an exercise date you have a choice between the continuation value with the original exercise strategy and the exercise value.
- With PI you take the bigger one.
- So will only increase value.

## More on PI

- If you continue you may get even more than the expected continuation value with the original value, since you will be using an improved strategy.
- Using induction, the overall value will increase.
- Less usefully, if you iterate enough times and have zero Monte Carlo error you get the perfect strategy.



## More on policy iteration

- Theoretically, if you ignore MC error will always improve lower bound.
- Main issue is speed.
- Very handy for getting a tight lower bound when testing...

## Practical policy iteration

- It turns out that with enough enhancements policy iteration can be made as fast as the unenhanced least-squares method.
- It has the great virtue of genericity – no product-specific coding is required.
- The main innovation is to use the non-iterated strategy as a control variate.

## PPI control-variate

- We simulate with the original strategy as well.
- If we reach a point where it says exercise then we exercise and repurchase.
- We stop when the iterated strategy says stop.
- If the non-iterated strategy is good then the two strategies will mostly agree.

## PPI and sub-simulations

- We need an estimate of the value of the control at each point where the two strategies disagree.
- This requires a sub-MC simulation.
- But we are already running this for the strategy!
- We also delta-hedge to minimize the size of these sub-simulations.

## Sub-simulations and bias

- Unfortunately, using the sub-sims for two purposes leads to bias.
- Essentially, one sub-simulation affects both the continuation estimate and the control and so the price.
- This problem is solvable, however.

# Sub-sims and numerical bias removal

- The easy solution is to use different simulations for the control.
- Seems slow but ...
  - At most doubles the time for a path: as long as variance is halved we gain.
  - If the original strategy is good, then not many exercise decisions disagree so actually much less than double the time.

## Sub-sims and analytical bias removal

- The bias is essentially arising from an option on the Monte Carlo error.
- We know how to price options.
- So compute the price of this option and subtract it.
- This works
  - Use Central limit theorem to assume normality
  - Bachelier formula to compute the price

## Non-useful paths

- When the decision is clear, we don't want to run many sub-paths.
- So only run sub-sims when the difference between exercise and unexercise estimate is small. (Broadie-Cao 2009)
- The main danger is that if the regression is poor, it won't be picked up.



## Adaptive Termination

- Run a few paths
- Quantify the probability that the decision given by using the sub-sim estimate is wrong by regarding the running estimate within the sub-sim as a normal random variable.
- Terminate when the probability is small or a ceiling has been reached.
- Paths used vary from 15 to 200.
- Will work even when the original estimate was very poor.

## Overall recommendations for first pass

- Work with breakable
- Use adaptive basis functions to do least squares on first pass
- Use double regression
- Exclude sub-optimal points from regression for exercise strategy

## Overall recommendations for second pass

- Use Delta hedging
- Use Quasi Monte Carlo with Brownian bridging and spectral decomposition
- Use policy iteration
  - Delta hedge the sub-simulations
  - Use the control variate
  - Use semi-analytic bias removal
  - Exclude provably sub-optimal points

## Example CMS Steepener

- Payout is a multiple of the 20 year rate minus the two year one, floored.
- Compare generic adaptive basis functions and policy iteration against tailor made choice due to Skovmand and Jorgensen of first forward rate, swap-rate to the end, 2 year swap rate and 20 year swap-rate

# CMS Steeper results for 6yr

|       | LS     | SO     | DR     | LSA    | SO<br>DR | SO<br>LSA |
|-------|--------|--------|--------|--------|----------|-----------|
| Adapt | -363.7 | -357.7 | -356.8 | -357.8 | -357.1   | -357.2    |
| SJ    | -370.5 | -362.8 | -359.8 | -360.2 | -359.2   | -361.5    |

Variation is 13 bps.

# CMS Steeper results for 10yr

|       | LS     | SO     | DR     | LSA    | SO<br>DR | SO<br>LSA |
|-------|--------|--------|--------|--------|----------|-----------|
| Adapt | -233.4 | -222.2 | -221.4 | -226.8 | -220.3   | -226.7    |
| SJ    | -247.7 | -243.8 | -231.1 | -249.4 | -231.7   | -247.8    |

Variation is 29 bps.

# CMS Steeper results for 20yr

|       | LS     | SO    | DR    | LSA    | SO<br>DR | SO<br>LSA |
|-------|--------|-------|-------|--------|----------|-----------|
| Adapt | -64.2  | -24.8 | -30.9 | -77.8  | -16.7    | -22.7     |
| SJ    | -143.4 | -72.0 | -89.1 | -191.8 | -60.0    | -89.9     |

Variation is 127 bps.

## Delta hedging: efficiency improvements

- Standard error reductions for 2000, 5000, 10000 first pass paths For 6yr, 10yr and 20 yr deals.

|     | 2000 | 5000 | 10000 |
|-----|------|------|-------|
| 6y  | 2.68 | 2.72 | 2.97  |
| 10y | 2.32 | 2.80 | 2.86  |
| 20y | 2.05 | 2.43 | 2.50  |

- Efficiency improvement is more like the square.



## Bermudan Swaption

- For Bermudan swaption 20 yr
  - Simple basis in LS: 934 bps
  - Adaptive, SO and DR: 1081 bps
  - PPI: 1090 bps
  - Upper bound: 1095 bps
- The upper bound is not the true price so true price is probably close to 1090.
- Time 8% of basic least squares method.

# Snowball

- 20 year snowball
  - Simple basis in LS: 502
  - Adaptive with SO and DR: 625
  - PPI: 639
  - Upper bound: 651
- The upper bound is not the true price so true price is probably close to 640.
- Time 16% of basic LS method

## Part VIII

Finding upper bounds for prices of early exercisable derivatives by Monte Carlo

# The need for upper bounds

- How do we know our lower bound is any good?
- Whilst Longstaff–Schwartz is theoretically convergent, it's not really a practical result.
- Best solution is to find an upper bound.
- If upper bound and lower bound are close then both must be good.

## Seller's price

- Seller cannot choose the exercise strategy.
- The seller has to have enough cash on hand to cover the exercise value whenever the buyer exercises.
- The seller's price is therefore the amount of money required to hedge against any exercise strategy.

## Maximal foresight

- The buyer could choose to exercise at random.
- $1/N$  chance of exercising at the maximum.
- In derivatives pricing we are supposed to cover no matter what happens.
- So we must hedge against someone exercising at the max.
- i.e. against someone exercising with maximal foresight.

## Seller's price continued

- Maximal foresight price:

$$\mathbb{E}(N_r^{-1}Z_r) = \mathbb{E}(\max N_t^{-1}Z_t)$$

- Clearly bigger than buyer's price.
- However, we have neglected the seller's ability to hedge.

## Hedging against maximal foresight

- Suppose we hedge as if the buyer were using an optimal stopping time strategy.
- At each date, either our strategies agree and we are fine
- Or
  - 1 buyer exercises and we don't
  - 2 buyer doesn't exercise and we do

In both of these cases we make money!



## The optimal hedge

- “Buy” one unit of the option to be hedged.
- Use optimal exercise strategy.
- If optimal strategy says “exercise.” Do so and buy one unit of option for remaining dates.
- Pocket cash difference.
- As our strategy is optimal at any point where strategy says “do not exercise,” our valuation of the option is above the exercise value.

## Rogers' / Haugh-Kogan method

- Equality of buyer's and seller's prices says

$$\sup_{\tau} \mathbb{E}(N_{\tau}^{-1} Z_{\tau}) = \mathbb{E}(\max_t N_t^{-1} (Z_t - P_t))$$

for correct hedge  $P_t$  with  $P_0$  equals zero.

- If we choose wrong  $\tau$ , price is too low = lower bound
- If we choose wrong  $P_t$ , price is too high = upper bound
- Objective: get them close together.

## Understanding the maths here

- Let  $H$  an arbitrary hedge with zero initial value: i.e. the price process of a self-financing portfolio.
- Its deflated value is therefore a martingale.
- Optional sampling theorem: says that a martingale stopped at a finite stopping time is a martingale.

# A general hedge

- So for any exercise strategy

$$\mathbb{E}(H_{\tau} N_{\tau}^{-1}) = 0.$$

- So if  $Z$  is the pay-off

$$\mathbb{E}(Z_{\tau} N_{\tau}^{-1}) = \mathbb{E}((Z_{\tau} - H_{\tau}) N_{\tau}^{-1}).$$

# Always an upper bound

- We therefore have with  $r$  a *random time*

$$\begin{aligned}\frac{D_0}{N_0} &= \sup_{\tau} \mathbb{E}(Z_{\tau} N_{\tau}^{-1}), \\ &= \sup_{\tau} \mathbb{E}((Z_{\tau} - H_{\tau}) N_{\tau}^{-1}), \\ &\leq \sup_r \mathbb{E}((Z_r - H_r) N_r^{-1}), \\ &= \mathbb{E}(\max((Z_r - H_r) N_r^{-1})).\end{aligned}$$

## Implementation approach 1

- We don't know the perfect hedge so take a family of portfolios that are similar.
- Optimize over that family of portfolios.
- For example, take the European swaptions for a Bermudan swaption and vary the notionals.
- Main issue is that we are optimizing a function that is evaluated by Monte Carlo.

## Implementing Rogers/Haugh Kogan

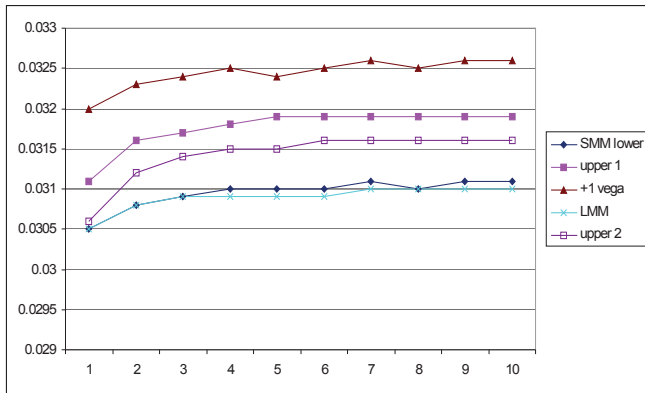
- We do two passes, first for optimizing.
- Second to get unbiased estimate of upper bound.
- The variance of the second pass should be very small since we have a good hedge.
- Optimal hedge characterizable by zero variance. So we can use optimization to search for zero variance portfolio instead.

## Example from Joshi-Theis

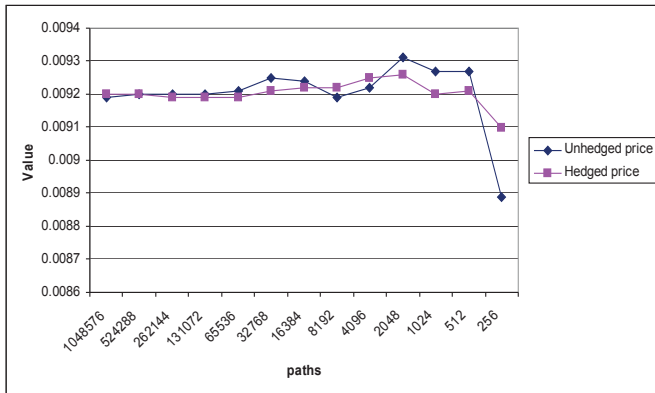
- Varied factors
- Kept European swaption vols fixed
- Used European swaptions for upper bounds
- Jäckel Method for lower bounds
- Did both LMM and SMM
- 5 year swaption.



## Example for Bermudan swaption



# Lower convergence



## Approximating the perfect hedge

- If we know the optimal exercise strategy, we know the perfect hedge.
- In practice, we know neither.
- Anderson-Broadie: pick an exercise strategy and use the product with this strategy as hedge, rolling over as necessary.
- Main downside: need to run sub-simulations to estimate value of hedge
- Main upside: tiny variance

## Sub-simulations

- The Anderson-Broadie method requires the value of the hedge for two reasons
  - At non-exercise points to assess the value of

$$Z_t - H_t.$$

- At exercise points, we need to reinvest so we need the continuation value.
- So apparently need to run a sub-simulation at every exercise point on every path

## Bias in Anderson–Broadie

- We use sub-Monte Carlo simulations to evaluate the hedge value.
- This leads to bias.
- However, the bias is upwards so we still have an unbiased estimate of an upper bound.

# Analyzing the bias

- We have to take a maximum along each path.
- Let  $x_j$  be the true values and  $e_i$  the errors.
- We have

$$\begin{aligned}\max(x_i) &= x_j, \text{ for some } j, \\ &= \mathbb{E}(x_j + e_j), \\ &\leq \mathbb{E}(\max(x_i + e_i))\end{aligned}$$

# Random numbers

- The random numbers for the outer path must be independent of those of the inner path.
- The random numbers for each inner path must be independent of those for each other inner path.
- Random numbers for different exercise times do not need to be independent within a path.

## Improving Anderson-Broadie

- Our upper bound is

$$\mathbb{E}(\max N_t^{-1}(Z_t - P_t)).$$

- The maximum could occur at a point where  $Z = 0$ , which makes no financial sense.
- Redefine  $Z$  to be equal to minus infinity at any point out of the money. (except at final time horizon.)
- Buyer's price not affected, but upper bound will be lower.



## Sub-simulations out-of-the-money

- If we redefine the pay-off to be minus infinity out of the money then we don't need the hedge value there so no sub-simulations.
- Added bonus: ITM sub-simulations take less time to run than out of the money ones

## Provable sub-optimality

- Suppose we have a Bermudan put option in a Black-Scholes model.
- European put option for each exercise date is analytically evaluable.
- Gives quick lower bound on Bermudan price.
- Would never exercise if value  $< \max$  European.
- Redefine pay-off again to be minus infinity.
- Similarly, for Bermudan swaption.

## Working with breakability directly

- As for lower bounds, we can work with the breakable contract directly.
- Rather than thinking of a single cash-flow arriving at time of exercise, we think of cash-flows arriving until the contract is broken.
- Equivalence of buyer's and seller's prices still holds, with same argument.
- Algorithm model independent and does not require analytic break values.

## Upper bounds for breakables

- Fix a break strategy.
- Price product with this strategy.
- Run a Monte Carlo simulation
  - Along each path accumulate discounted cash-flows of product and hedge.
  - At points where strategy says break. Break the hedge and “Purchase” hedge with one less break date, this will typically have a negative cost. And pocket cash.
  - Take the maximum over set of break dates of the cumulative difference of cash-flows.

## Jamshidian's method

- An alternative approach to upper bounds by Monte Carlo is due to Jamshidian.
- Suppose a Bermudan option always has positive pay-off at the final time then its value is always positive.
- We can therefore take it as numeraire.
- The ratio of the value of the pay-off to the value of the derivative is always less than or equal to one at exercise dates (since we can always exercise)

# Jamshidian valuation formula

- Let  $N$  be the numeraire which is also the value of the option.
- We have

$$\begin{aligned} D_0 &= N_0 \mathbb{E}(\max(Z_t/N_t)), \\ &= N_0 \mathbb{E} \left( \frac{N_T}{N_T} \max(Z_t/N_t) \right). \end{aligned}$$

- We can regard this as the value of a look-back option paying at time  $T$  :

$$N_T \max(Z_t/N_t).$$

# Jamshidian's equation

- Change of numeraire yields:

$$D_0 = N_0 \mathbb{E}_N \left( \frac{H_n}{N_n} \max_j \frac{Z_j}{H_j} \right).$$

- where  $H$  is the hedge consisting of the derivative itself, and on exercise purchasing extra units of itself.
- And  $N$  is any numeraire

## Jamshidian's method

- For the correct hedge, we get equality for a general  $H$  we get an upper bound:

$$D_0 \leq N_0 \mathbb{E}_N \left( \frac{H_n}{N_n} \max_j \frac{Z_j}{H_j} \right).$$

- So pick an  $H$  close to the derivative and see what you get.



## Extending Jamshidian

- Similarly to Rogers' method, we can use the derivative itself with a sub-optimal strategy as the hedge and still get an upper bound.
- Involves sub-MC simulations.
- We can exclude out of the money points as before.

## Dealing with zero

- Either use analytic value at second final time frame
- Or a limiting argument shows that the zeros can be cancelled:

$$D_0 = N_0 \mathbb{E}_N \left( \max \left( \frac{Z_n}{N_n}, \frac{H_n}{N_n} \max_{j < n} \frac{Z_j}{H_j} \right) \right).$$

## Rogers vs Jamshidian

- Jamshidian does not apply to the breakable case: final value must be non-negative.
- Rogers has lower variance.
- Either can be better.
- Only a tiny amount of extra effort to run both at once, if you are running one of them.

## Bermudan Factor dependence: history

- Longstaff Santa Clara, Schwartz: “Throwing away a billion dollars”
- Anderson-Andreasen : factors make little difference and if anything decrease price
- Choy, B., Dun, T., Schlgl, E., “Correlating Market Model,” Risk: correlation has little effect

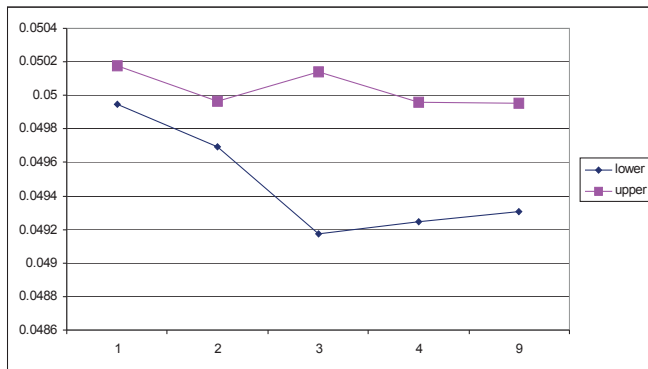
# Elementary calculus

- A partial derivative is only meaningful in terms of what else is held fixed.
- Issues are
  - What you calibrate to
  - Do you have an upper bound
  - What shapes of vols you use
- LSS : used 1 factor flat vols
- Anderson–Andreason: calibrated to entire swaption matrix.

## Assessing factor dependence

- We have good upper bounds
- We have good lower bounds
- We have simultaneous calibration to caplets and co-terminal swaptions.
- We can assess how much difference the number of factors makes.
- Vary number of factors whilst holding caplet and coterminal swaptions fixed.
- Most time homogeneous fit

# 10 NC 1 Bermudan factor dependence

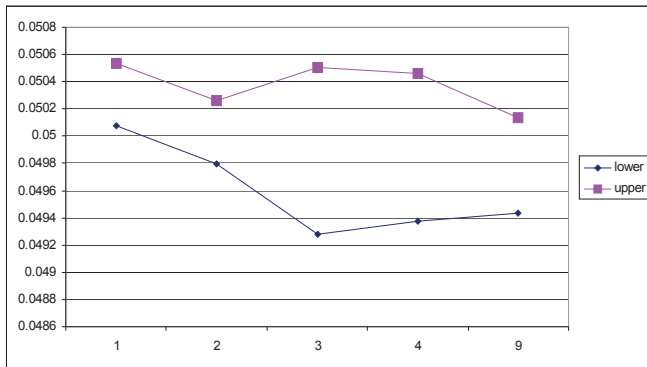


## Time dependence of vols

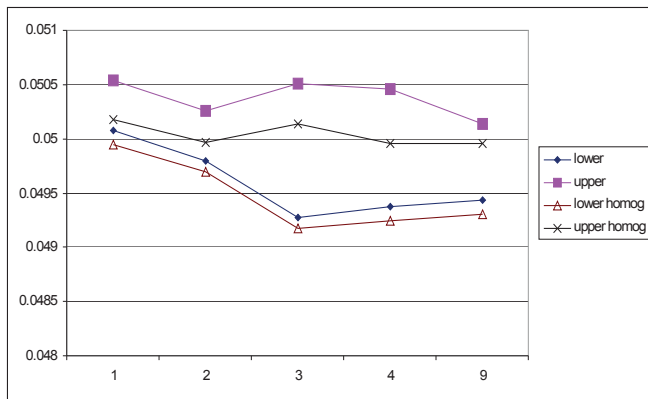
- Suppose we start with flat vols i.e. model vols for swap-rates are constant rather than time homogeneous.
- Carry out the same calibration algorithm
- What happens?



# 10NC1 Bermudan factor dependence flat vols



# 10NC1 Bermudan factor dependence flat versus time homogeneous



# Snowball cancellable note

- Path-dependent coupon, cancellable semi-annually. Coupons as follows:

$$K_i = I, \text{ for } i = 0, \dots, h,$$

$$K_i = (K_{i-1} + (A_i - f_i(T_i))\tau_i)_+, i > h$$

- Our, example:  $h = 1$  and  
 $I = 7\%$ ,  $A_i = 3\%, 3\%, 3.25\%, 3.25\%, 3.5\%, \dots$

# Snowball basis functions

- $f_l(T_j)$ ,
- $SR_{j+1}(T_j)$  (the swap-rate starting at the next tenor date and running to final tenor date),
- $FL_j(T_j)$  (the value of the floating leg of the swap running from  $T_j$  to the final tenor date, also evaluated at  $T_j$ ),
- the current snowball coupon,
- quadratic polynomials in these.

# BKS approach

- Basis functions
  - $f_j(T_j)$ ,  $SR_{j+1}(T_j)$  and current coupon.
  - quadratics in these
- Found these were bad so suggested using policy iteration on top.

## Example cases

We (Beveridge and Joshi) studied two cases:

### 1 Flat

- Flat curve at 3.5%
- Flat vol 20%
- Full factor model

### 2 Sloped

- Ascending curve
- Higher vols and displacement of 1.5%

## Results case 1: BKS basis functions

|        | Lower  | Upper  | Gap   | PI     | Vega  |
|--------|--------|--------|-------|--------|-------|
| LS     | 77.37  | 119.88 | 42.51 | 103.65 | 26.10 |
| LSA    | 92.65  | 111.11 | 18.44 | 108.35 | 20.42 |
| LS, +  | 97.64  | 111.18 | 14.12 | 107.73 | 22.91 |
| LSA, + | 100.19 | 110.30 | 10.11 | 108.11 | 21.05 |

LS = least squares , A = Anderson, + = exclude sub-optimal points

## Results case 1: new basis functions

|       | Lower  | upper  | Gap  | PI     | Vega  |
|-------|--------|--------|------|--------|-------|
| LS    | 91.04  | 115.00 | 24.0 | 106.68 | 24.51 |
| LSA   | 101.15 | 109.36 | 8.21 | 108.37 | 20.28 |
| LS,+  | 103.80 | 109.39 | 5.59 | 108.56 | 22.58 |
| LSA,+ | 105.67 | 109.19 | 3.52 | 108.41 | 20.66 |

LS = least squares , A = Anderson, + = exclude sub-optimal points



## Results case 2: BKS basis functions

|       | Lower  | Upper  | Gap   | PI     | Vega  |
|-------|--------|--------|-------|--------|-------|
| LS    | 73.55  | 144.67 | 71.12 | 117.21 | 25.24 |
| LSA   | 104.15 | 131.68 | 27.53 | 127.19 | 19.56 |
| LS,+  | 104.03 | 131.49 | 27.46 | 124.28 | 23.03 |
| LSA,+ | 110.64 | 129.85 | 19.21 | 125.31 | 20.69 |

LS = least squares , A = Anderson, + = exclude sub-optimal points

## Results case 2: new basis functions

|       | Lower  | Upper  | Gap   | PI     | Vega  |
|-------|--------|--------|-------|--------|-------|
| LS    | 102.22 | 135.11 | 32.89 | 123.92 | 23.78 |
| LSA   | 114.35 | 127.97 | 13.62 | 125.53 | 20.07 |
| LS,+  | 117.64 | 127.71 | 10.07 | 125.52 | 22.48 |
| LSA,+ | 121.61 | 126.99 | 5.38  | 125.42 | 20.37 |

- Over conclusion: PI is good but LSA,+ with the new basis functions is good enough.
- Or use practical policy iteration...

## Policy iteration and upper bounds

- The Andersen–Broadie upper bound hedge and the control variate are the same.
- They require the same sub-simulations.
- So we can run both at once.
- We can also use Delta hedging to reduce the number of paths needed for the upper bounds.

## Part IX

### Pathwise Greeks and adjoints in the LMM

## Before or after discretization

- We can differentiate to get a model Greek before or after we discretize the model.
- Differentiation before is more theoretically correct but requires very detailed knowledge of the LMM density which is not available.
- Differentiation after really means differentiating the numerical scheme.

# Model Greeks

- A cash-flow is a function of the prevailing rates at the time of its determination.
- These rates will be determined by
  - The inputs Gaussians from the random number generator
  - The initial rates
  - The pseudo-root elements used to evolve to the time frame
  - The choice of numerical scheme

# Chaining

- A change to one of these in a time-step affects all subsequent time steps.
- Differentiation therefore affects not just a given time-step but involves multiple applications of chain rules.
- But this is not hard!

# Notation

- Suppose we have a cash-flow  $C$ , at time  $t_k$  which is determined by

$$X_k = (f_1(t_k), f_2(t_k), \dots, f_n(t_k)).$$

- We can write

$$C = g(X_k).$$

- For deltas, we want the derivative with respect to the initial rates, i.e  $X_0$ .



# The differentiation for Deltas

- Chaining we get,

$$\frac{\partial g}{\partial X_k} \frac{\partial X_k}{\partial X_{k-1}} \frac{\partial X_{k-1}}{\partial X_{k-2}} \cdots \frac{\partial X_1}{\partial X_0}.$$

- The first term must be reasonably easy since pay-offs can never be very complicated...
- The remaining terms are derivatives of the numerical scheme.

# State-dependence

- In the LMM, the terms

$$\frac{\partial X_j}{\partial X_{j-1}}$$

are state-dependent.

- They must be computed on every step of every path.

# Naive computational order

- We compute  $k$  matrices, we multiply them together.

- This is of order

$$kn^3$$

- With  $k$  typically as big as  $n$ .

# Rebracketing

- We can rewrite the product as

$$\left( \cdots \left( \left( \frac{\partial g}{\partial X_k} \frac{\partial X_k}{\partial X_{k-1}} \right) \frac{\partial X_{k-1}}{\partial X_{k-2}} \right) \cdots \frac{\partial X_1}{\partial X_0} \right).$$

- Each multiplication is now  $n^2$  so total order is

$$kn^2.$$

## Further improvements: Giles–Glasserman

- In the LMM, the computational order of a time-step is  $nF$  not  $n^2$ .
- This suggests that the Jacobians will have special structure that reduces the computations necessary.
- This turns out to be true, and the computation can be done in  $knF$  time.
- With multiple cash-flows, we aggregate as much as possible.

## General theorem: Griewank

- if you can compute a number using  $M$  elementary operations,
- you can compute all its derivatives with  $4M$  elementary operations.
- this is amazing!

# Elementary operations

Any numerical computer program can be reduced down to lists of simple commands such as

- $y_1 + = y_2,$
- $y_3 * = y_4,$
- $y_5 - = y_6,$
- $y_7 := \exp(y_8),$
- $y_9 := \cos(y_{10}).$

Note: we outlaw *if* and discontinuous functions.

# Differentiation of elementary operations

- Suppose we know the gradient of  $f(X_I)$  which we denote  $v$
- and  $X_I = e(X_{I-1})$  with  $e$  an elementary operation then

$$\nabla_{X_{I-1}} f(X_I) = v J_e$$

- where  $J_e$  is the Jacobian of  $e$ .
- $J_e$  will be very simple.



## Right multiplication by elementary Jacobians

- We don't compute  $J_e$ .
- Instead for each class of operations, we compute the effect of right multiplication by  $J_e$ .
- For elementary operations, this is very simple.
- It will only affect one or two entries of  $v$ .
- It will often be simpler than the operation.

## Example of elementary Jacobian multiplication

- We know  $f(x, y)$  and  $\nabla_{x,y} f(x, y)$ .
- We want to compute  $\nabla_{x,z} f(x, e^z)$ .
- $x$  entry doesn't change.
- $\frac{\partial f}{\partial z} = \frac{\partial f}{\partial y} e^z$
- So simply have to multiply the second entry by  $e^z$ .
- We already needed to compute  $e^z$  for computing the value so the additional work is tiny.

# Forwards and backwards

- We first do the forward computation of the value.
- We store the intermediate details of the computation.
- We then work backwards from the end.
- We apply the elementary Jacobians one at a time.
- we don't actually generate the Jacobians but simply work out their effects.
- The backwards (or adjoint) calculation can either be coded manually or generated automatically.
- The extra effort is at most 4 times the original effort in both cases.

# Computing the products a la GG

- Let

$$v_0 = \frac{\partial g}{\partial X_k}.$$

- and

$$v_{j+1} = v_j \frac{\partial X_{k-j}}{\partial X_{k-j-1}}.$$

- We want a quick algorithm for computing these. Here
- $X_k$  is the forward rate  $f_k$ .
- We use the log-Euler approximation

# The numerical scheme

- We have

$$f_i(r) = f_i(r-1)e^{-0.5C_{ii} + \mu_i(f(r-1)) + \sum a_{ij}Z_j},$$

- where

$$\mu_i = \sum_{j \leq i} C_{ij} \frac{\tau_j f_j}{1 + \tau_j f_j}.$$

# Its derivative

- We get  $\frac{\partial f_i(r)}{\partial f_j(r-1)} = \delta_{ij} \frac{f_i(r)}{f_i(r-1)} + f_i(r) \frac{\partial \mu_i}{\partial f_j}.$

- and

$$\frac{\partial \mu_i}{\partial f_j(r-1)} = C_{ij} \frac{\tau_j}{(1 + f_j \tau_j)^2}.$$

# The product

If  $v = w \frac{\partial f(r)}{\partial f(r-1)}$ , then

$$\begin{aligned} v_j &= \sum_{i \geq j} w_i \frac{\partial f_i(r)}{\partial f_j(r-1)}, \\ &= w_i \frac{f_i(r)}{f_i(r-1)} + \sum_{i \geq j} f_i(r) w_i C_{ij} \frac{\tau_j}{(1 + \tau_j f_j(r-1))^2}. \end{aligned}$$

# Quick algorithm

- If  $h_i = w_i f_i(r)$ , then

$$\sum_{i \geq j} h_i C_{ij} = \sum_{i \geq j} \sum_{l=1}^F h_i a_{il} a_{jl}.$$

- Or

$$\sum_{l=1}^F a_{jl} \sum_{i \geq j} h_i a_{il}.$$



## Quick algorithm II

- The inner sums are incremental so can be computed in order  $nF$  time for all values of the indices.
- So the whole thing is computable in order  $nF$  time.

## Model vegas

- We want to perform a similar computation for vegas.
- Or indeed for any other possible parameter of the model.
- A model vega is effectively a derivative with respect to an entry in a pseudo-root matrix.

# Vega Set-up

- We can write

$$X_k = F_{k-1}(X_{k-1}, \theta, Z_k),$$

- with  $\theta$  denoting all the parameters of the model including, for example, the pseudo-root elements, and  $Z$  the Gaussians.
- Differentiation is tricky because it cascades through to all higher terms.

# Cascade

- Differentiating, gives

$$\frac{\partial X_k}{\partial \theta} = \frac{\partial X_k}{\partial X_{k-1}} \frac{\partial X_{k-1}}{\partial \theta} + \frac{\partial F_{k-1}}{\partial \theta}.$$

- Repeating, it becomes

$$\frac{\partial X_k}{\partial X_{k-1}} \left( \frac{\partial X_{k-1}}{\partial X_{k-2}} \frac{\partial X_{k-2}}{\partial \theta} + \frac{\partial F_{k-2}}{\partial \theta} \right) + \frac{\partial F_{k-1}}{\partial \theta}.$$

# All the way to zero

- We eventually get

$$\sum_{j=1}^k \left( \prod_{l=1}^j \frac{\partial X_{k-l+1}}{\partial X_{k-l}} \right) \frac{\partial F_{k-j}}{\partial \theta},$$

- And multiplying by the pay-off derivative, we get

$$\sum_{j=1}^k \frac{\partial g}{\partial X_k} \left( \prod_{l=1}^j \frac{\partial X_{k-l+1}}{\partial X_{k-l}} \right) \frac{\partial F_{k-j}}{\partial \theta}.$$

# Computing

- Everything but the last term was already needed for the Delta computation.
- So the additional work is not so great.
- We still need to compute

$$\frac{\partial F_r}{\partial \theta},$$

- And determine what we mean by  $\theta$  !

## Elementary vegas

- The main parameters determining the evolution in the LMM are the elements of the pseudo-root matrices of the covariance matrices across each time step.
- We shall call the derivative with respect to one of these an *elementary vega*.
- There are an awful lot of these, since we have a matrix for each time step.

# Computing elementary vegas

- We need to differentiate the numerical scheme,  $F_{r-1}$ , used to get  $f(r)$  from  $f(r-1)$
- with respect to element  $a_{jk}$  of step  $s$ .
- This will clearly be zero unless  $r = s$ .
- We now focus on that case.



# Differentiating the scheme

- We obtain

$$f_i(r) \left( -\frac{1}{2} \frac{\partial}{\partial a_{jk}} C_{ii} + \frac{\partial}{\partial a_{jk}} \mu_i + \frac{\partial}{\partial a_{jk}} \sum_l a_{il} Z_l \right).$$

- We then have to each of these terms individually.

# First term

- We have

$$C_{ii} = \sum_{l=1}^F a_{il}^2,$$

- So we get

$$\frac{\partial}{\partial a_{jk}} C_{ii} = \delta_{ij} 2a_{ik}.$$

## Second term

- We must differentiate

$$\sum_{s \leq i} C_{is} \frac{f_s \tau_s}{1 + f_s \tau_s}.$$

- And the derivative of  $C_{is}$  is

$$\frac{\partial}{\partial a_{jk}} \sum_{l=1}^F a_{il} a_{sl} = \delta_{ij} a_{sk} + \delta_{sj} a_{ik}.$$

# Drift derivative

- If  $i < j$ , we get zero, for  $i = j$ :

$$\frac{\partial}{\partial a_{ik}} \mu_i = 2a_{ij} \frac{f_i \tau_i}{1 + f_i \tau_i};$$

- For  $i > j$ ,

$$\frac{\partial}{\partial a_{jk}} \mu_i = a_{ik} \frac{f_j \tau_j}{1 + f_j \tau_j};$$

## Third term

- We need

$$\frac{\partial}{\partial a_{jk}} \sum a_{il} Z_l.$$

- This is zero unless  $i = j$  and then we get

$$Z_k.$$

- The derivative is therefore fully stochastic.

## Putting it together

- We develop the path going forward through time, storing everything necessary to compute the Jacobians, gathering cash-flows as we go.
- We then multiply the derivatives of the cash-flows by the Jacobians working backwards aggregating as we go.

## Beyond Giles-Glasserman

- So far we have outlined work up to 2005
- Giles–Glasserman 2005: first application to derivatives pricing; the LIBOR market model using an Euler drift approximation
- What about
  - Bermudan derivatives
  - are further speed ups possible?
  - second derivatives/Hessians/Gammas? (GG too slow)
  - discontinuous pay-offs?
  - more accurate drift approximations?
  - swap-rate market models,

## What about the presence of an exercise strategy?

- Traditional to fix exercise time when path bumping.  
(Piterbarg)
- If we take small  $\epsilon$  limit of bump, then get derivative that ignores changes in the exercise strategy.
- The value at an exercise date is the maximum of the exercise value and the continuation value.
- It is therefore differentiable except at the point where the two agree.
- At that point, it is Lipschitz continuous.
- This is enough to apply the pathwise method.



## Bermudans and adjoints

- Take each cash flow generated as it occurs and differentiate it, ignoring the exercise strategy's dependence, and proceed as before.
- This is correct for Deltas.
- This is an approximation for Vegas but the error is slight: the difference is that the exercise boundary does change with volatility.
- This is badly wrong for Gammas!

## Pathwise gammas

- Consider simple case of a single driftless forward rate.
- Gamma of a caplet.
- If we ignore the exercise time dependence then we get zero since away from the derivative discontinuity the value is linear.
- The true value is not zero.
- All the value comes from the kink at the exercise boundary.
- Some smoothing or other solution is necessary.

# Hessians

- The GG algorithm is order  $n^3$  per step for Gammas, this is too slow. Can we do better?
- The Hessian is a matrix of size  $n \times n$ .
- Applying an elementary operations affects an entire row and column not just one element.
- Consider again  $f(x, e^z)$ .
- The cross derivatives  $f_{xz}$  have to be computed as well as  $f_{zz}$ .
- Only  $f_{xx}$  does not change.

## Hessians in backwards mode

- Let  $M$  be the maximum number of *state variables* needed in the computation at a single time.
- State variable: a quantity that is affected by changing the inputs and will affect the output.
- Note does not include random variables or pseudo-root elements if doing forward rate gammas.
- We then deal with Hessians with  $M$  rows and columns at intermediate stages in the computation.
- Main result (Joshi–Yang 2011) for a given class of elementary operations, there exists constants  $A$  and  $B$  such that the additional time taken to compute the Hessian is bounded by  $AM + B$  times the taken for computing the value.

# Hessians and Monte Carlo

- In Monte Carlo applications, on each path the computation is generally not that complex.
- In the LIBOR market model, we can achieve  $M = n + F + 3$  with  $F$  the number of factors.
- Note that for other methods, eg, PDE grid,  $M$  may be huge, however.
- $A$  and  $B$  are typically small.
- For the LMM, the Hessian computation typically takes  $n/3$  times as long as the price to compute all of the  $n(n+1)/2$  Hessian entries.

## Bermudan smoothing

- True value is Lipschitz continuous at the boundary but not differentiable.
- The sub-optimal-exercise-strategy determined value has a jump, since the true continuation value will not match the exercise value at the point of change.
- Easiest solution instead of making exercise all or nothing, exercise a fraction depending on closeness to estimated boundary.
- This makes the value  $C^2$  but slightly less optimal and the pathwise method applies.

## Coordinate shifts

- Automatic differentiation differentiates an algorithm,  $A$ , to give a new algorithm,  $D$ , for differentiation.
- Changing  $A$  will change  $D$  even when the final numbers do not change.
- We can explore rewrites of the algorithm that reduce speed further but give the same values on every path.
- Denson–Joshi (2009) using log coordinates for the LMM achieves an extra speed up of 20%.

## Pathwise adjoint predictor-corrector

- Giles–Glasserman differentiated the log Euler scheme.
- We usually price in the LMM using predictor-corrector (PC) or iterative predictor corrector (IPC).
- So differentiate the PC/IPC scheme instead.
- These work and can be done with the same computational order.
- More accurate.
- Denson–Joshi 2009/2011, Joshi–Wiguna 2010



## Generic market models

- The LIBOR market model evolves forward rates.
- Swap-rate market models work with swap-rates. (Jamshidian 1997)
- Can the same techniques be applied?
- Yes: Joshi-Yang (2009). Computational order is the same as for the LMM.

## References

- Denson, Joshi, Flaming Logs, *Wilmott Journal* 2009, No 5-6, 259 – 262
- Giles, Glasserman, Smoking Adjoints, *Risk* 2005
- Griewank, Walther, Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation, Second Edition
- M.S. Joshi, C. Yang, Algorithmic Hessians and the Fast Computation of Cross-Gamma Risk, *IIE Transactions*, Volume 43, Issue 12, 2011, 878–892
- Joshi, Yang, Fast Delta Computations in the Swap-Rate Market Model, to appear in *Journal of Economic Dynamics and Control*

## Part X

Computing model and market Greeks in the LIBOR market model using proxies and combinations

# The importance of Greeks

- Much of the work of a quant is really about computing Greeks.
- The market will find a price for a security but it will not find a sensitivity.
- Sensitivities are important because
  - Determine risk numbers,
  - Determine hedges.

# Naive approach

- Bump market data
- Recalibrate
- Revalue
- Virtues:
  - Easy to implement
  - Generic
  - Reflects what a trader really regards as a Greek

# Naive approach implementation

- Calibration must be stable
- Better to use centred differencing:

$$\frac{f(\theta + h) - f(\theta - h)}{2h}.$$

- Get error that is  $O(h^2)$  rather than  $O(h)$ .
- However, centred differencing may not be what the trader wants.

# Random numbers

- Our estimate is subject to Monte Carlo error so we get

$$\frac{1}{2h} (f(x+h) + \epsilon_h^1 - f(x-h) - \epsilon_h^2).$$

- If we use different random numbers for the two passes, our error has standard deviation

$$\frac{1}{2h} \sqrt{\sigma_1^2 + \sigma_2^2}.$$

# Synchronizing random numbers

- For  $h$  small, we get very large variance.
- If we use the same random numbers for each simulation, the two errors will be highly correlated, so

$$\epsilon_h^1 - \epsilon_h^2$$

is much smaller.

- We can do this by resetting to the same seed rather than by storing the numbers.



# Problems with the naive approach

- Discontinuities are magnified.
- If we use the same random numbers, we are essentially differencing on a path-by-path basis:

$$\begin{aligned} \frac{1}{2h} \sum_i G(f^i(\theta + h)) - G(f^i(\theta - h)) \\ = \sum_i \frac{1}{2h} G(f^i(\theta + h)) - G(f^i(\theta - h)). \end{aligned}$$

# Bumping and digital caplets

- So most paths give zero.
- A few paths give

$$\pm \frac{1}{2h} P_1$$

- This leads to huge variance.
- Even worse for second derivatives.

# Interpretation

- The problem is that as

$$h \rightarrow 0+,$$

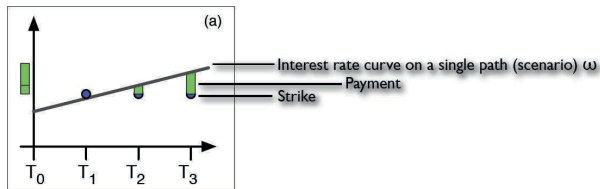
the fraction inside the sum does not converge to the derivative.

- In truth, the derivative is a delta function which does not exist in the conventional sense of a derivative.

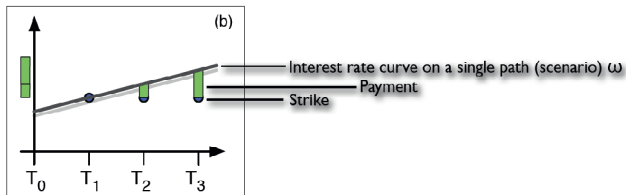
## Example: auto-cap

- This can be a problem even with apparently benign products.
- A small bump can make a caplet shift from being out of the money to being in the money and hence cause another caplet to have zero value.
- Only a small fixed number pay.

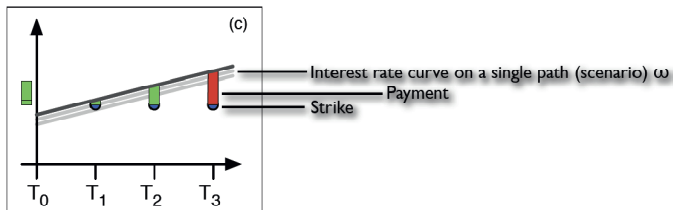
# Auto-cap



## Auto-cap 2

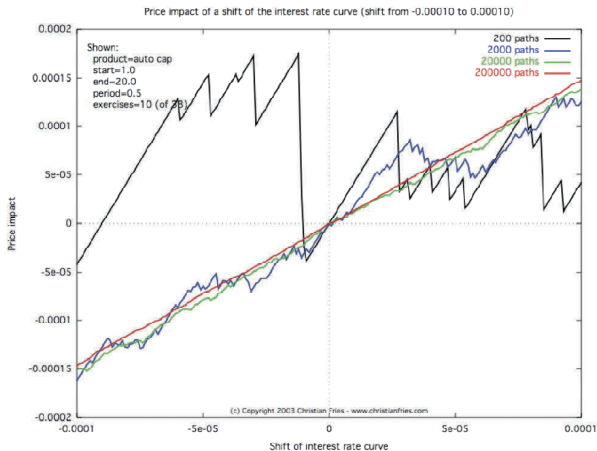


## Auto-cap 3



# Auto-cap 4

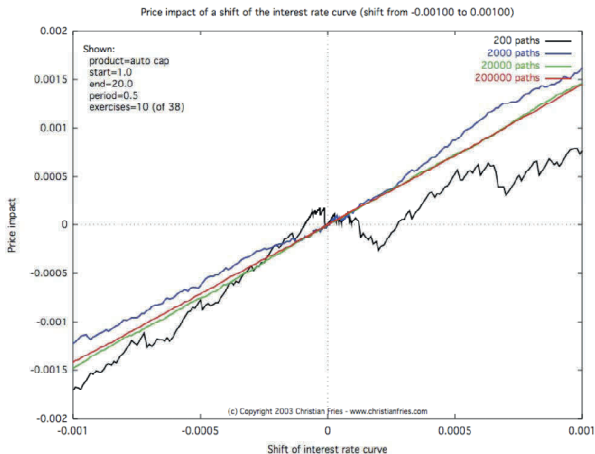
## Example: AutoCap Sensitivities: 1 bp shift





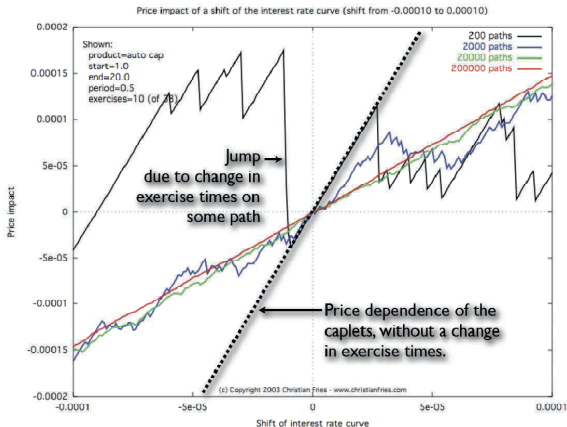
# Auto-cap 5

## Example: AutoCap Sensitivities: 10 bp shift



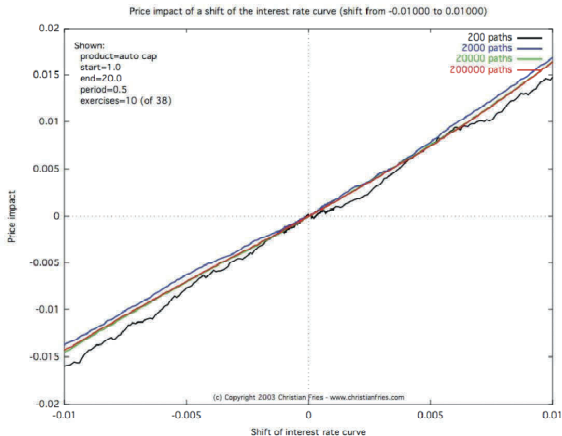
# Auto-cap 6

## Example: AutoCap Sensitivities: 1 bp shift



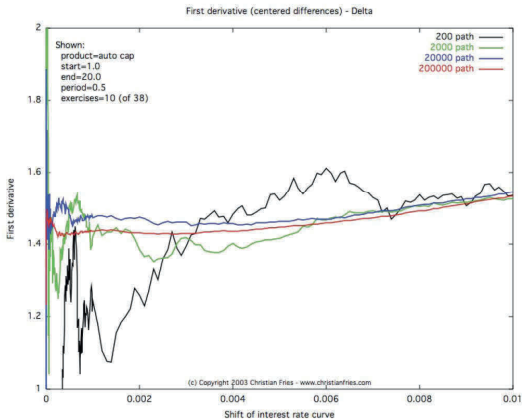
# Auto-cap 7

## Example: AutoCap Sensitivities: 100 bp shift



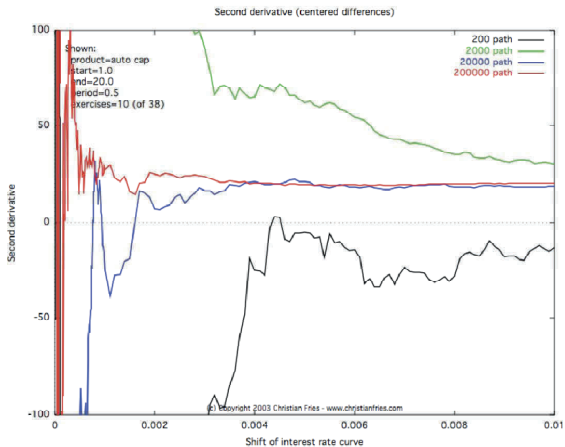
# Auto-cap 8

## Example: AutoCap Sensitivities: Delta 100 bp



# Auto-cap 9

Example: AutoCap Sensitivities: Gamma 100 bp



# The TARN

- Recall TARN pays coupons until a target level is reached.
- Coupons are inverse floating, so they can hit zero and then the bank pays nothing and receives floating until deal maturity.
- With upwards sloping yield curve these properties are magnified.
- Typically, one critical rate so effectively a digital caplet.

## Standard methodologies

There has been a large amount of work done on Greeks in Monte Carlo models:

- Pathwise method as we discussed last session.
- Likelihood ratio method
- Malliavin calculus

## Model versus market Greeks

- Sophisticated methodologies for Greek computation generally compute the sensitivity to shifting a parameter of the model.
- Traders and risk managers want the sensitivity to changes in market data.
- These are NOT the same thing.



# Model sensitivity

- 1 Price in model.
- 2 Bump one parameter.
- 3 Reprice in model.
- 4 Take difference.

# Chain rule solution

- If we know
  - the sensitivity of the model to all parameters
  - the sensitivity of the parameters to the market data
- Then we can use the chain rule to compute the sensitivities to market data
- But cumbersome in practice.
- We will return to this point...

## The path-wise method (pure)

- We write our integral in such a way that the parameter dependence is in the pay-off.
- We differentiate.
- Requirements
  - Intimate knowledge of SDE
  - But not simulation scheme
  - Handcrafted to pay-off
  - Handcrafted to parameter.

## The path-wise method (approximate)

- We write our integral in such a way that the parameter dependence is in the pay-off and approximate with an SDE scheme.
- We differentiate.
- Requirements
  - Intimate knowledge of Solution scheme
  - But not SDE
  - Handcrafted to pay-off
  - Handcrafted to parameter.

## Further pathwise properties

- We do not modify paths
- So we can compute many Greeks at once.
- Standard formulation requires the derivative of the pay-off to be a function
- i.e. requires Lipschitz continuity of the pay-off.
- No discretization bias.

# Pathwise versus finite differences

- Take a pay-off for which the approximate path-wise method applies.
- Compute Greek using
  - Approximate pathwise method
  - Finite differences on that parameter
- Variance is the same!
- Pathwise is just the limit of finite differences.

# Finite difference limit

- So when approximate path-wise method applies it the same as finite differences:

$$\begin{aligned} \lim_{h \rightarrow 0} \sum_i \frac{1}{2h} G(f^i(\theta + h)) - G(f^i(\theta - h)) \\ = \sum_i \frac{\partial}{\partial \theta} G(f^i(\theta)). \end{aligned}$$

- But only requires 1 simulation.
- And no bias.

# Pathwise and discontinuous pay-offs

- The derivative of a Heaviside function is a delta function in distribution sense.

$$H'(x) = \delta_0(x)$$

- Where

$$H(x) = \begin{cases} 1 & \text{for } x \geq 0, \\ 0 & \text{for } x < 0 \end{cases}$$



# Decomposition

- Integration against a delta function is easy!
- By definition, it is the value of the function.
- So write the pay-off as a continuous function plus Heaviside functions.
- Evaluate derivatives of Heavisides analytically.
- Evaluate remaining functions by pathwise.
- It works but requires a lot of handcrafting

# Likelihood ratio

- The fundamental idea is to put the entire dependence on the parameter into the density.
- So we write after change of variables

$$\int G(f) \psi(f, \theta) df.$$

- We differentiate under the integral sign.

# Differentiation

- We get

$$\begin{aligned}\frac{\partial}{\partial \theta} \int G(f) \psi(f, \theta) df &= \int G(f) \frac{\partial \psi}{\partial \theta}(f, \theta) df. \\ &= \int G(f) \frac{1}{\psi} \frac{\partial \psi}{\partial \theta}(f, \theta) \psi(f, \theta) df.\end{aligned}$$

# LR formula

- So we get

$$\int G(f) \frac{\partial \log \psi}{\partial \theta}(f, \theta) \psi(f, \theta) df.$$

- Just multiply pay-off by an extra term on each path and we are done.

# LR features

- Likelihood ratio is product independent.
- Requires detailed information on the density or on the approximate density implied by the simulation scheme.
- Sensitivities model not generic
- Applies to non-smooth pay-outs.
- Variances can be large.
- Unbiased.

## Implicit assumption

- A further assumption is that the support of the density does not change with the parameter.
- Whilst this is apparently innocuous, this is, in fact, a big problem with reduced factor LMM.
- Remember speed is proportional to factors so we don't want to do full factor.

# Support of discretized LMM density

- If we take a single-step in the LIBOR market model using an Euler scheme then we have

$$\log f_i(t) = \log f_i(s) + \mu_i(s) - 0.5C_{ii} + \sum_{j=1}^F a_{ij}Z_j.$$

- Only  $F$  sources of randomness so image is  $F$ -dimensional.

# Perturbation

- If we perturb parameters, the image sub-manifold will change and will generally be disjoint from the previous one.
- Same issues with predictor-corrector: image is  $F$ -dimensional and curved in log space.
- So LR Method does NOT apply!



## Partial proxy method

- The image is a sub-manifold of  $F$  dimensions so its projection onto an  $F$  dimensional plane will be onto.
- So although the sub-manifolds will be disjoint, their projections will be the same.
- What this means is that we can do likelihood ratio in  $F$  dimensions whilst going pathwise/finite difference in the rest.

# Proxy constraint

- Most discontinuities are caused by a single rate.
- Eg TARN if a reference rate is below a certain level, product terminates, otherwise continues.
- So we generally only need to fix one rate to kill discontinuities.
- This could be a LIBOR rate or a swap-rate.

# Achieving the constraint

- Before step
  - Unperturbed rate was  $\log(f_i(s))$ .
  - Perturbed rate was  $\log(\tilde{f}_i(s))$ .
- We therefore mean-shift the Gaussian underlying rate  $i$  by

$$\frac{\log f_i(s) - \log \tilde{f}_i(s)}{\sigma}$$

with  $\sigma$  the standard deviation across the step

## Likelihood ratio

- Mean shifting a Gaussian leads to a likelihood ratio weight on the simulation
- But this is just usual Girsanov-style ratio of normal densities

$$\frac{e^{-z^2/2}}{e^{-(z-\mu)^2/2}}.$$

# Multi-factor

- We will typically have

$$\log \tilde{f}_i(s) + \mu + \sum_j a_{ij} Z_j.$$

- So can shift any of the Gaussians.
- Minimal change is to shift in direction of maximal impact, i.e., by the vector  $v$  with

$$v_j = (a_{ij})$$

## How it works

- Calibrate model
- Recalibrate model after bump
- Run unperturbed simulation and perturbed simulation(s) together.
- At each step compute mean shift to make the important rate in agreement with unperturbed case.
- Accumulate likelihood ratios along path

# Speed ups

- If the deflated value only depends on the constrained rate
- Then the pay-off is the same for perturbed and unperturbed cases.
- So only need to compute the likelihood ratio.
- This only depends on the bump and the constraint, but not the product.

## Partial proxy Greek features

- Generic: only need to specify rate constraint at each step
- Need good understanding of simulation scheme
- Pay-off independent
- Small bias
- Discontinuous pay-offs
- Do not need to understand SDE or density.



## Why only constrain 1 rate?

- Path-wise/finite difference gives lower variances than likelihood ratio
- So use likelihood ratio in only the 1 direction that matters and pathwise/finite difference in the others.

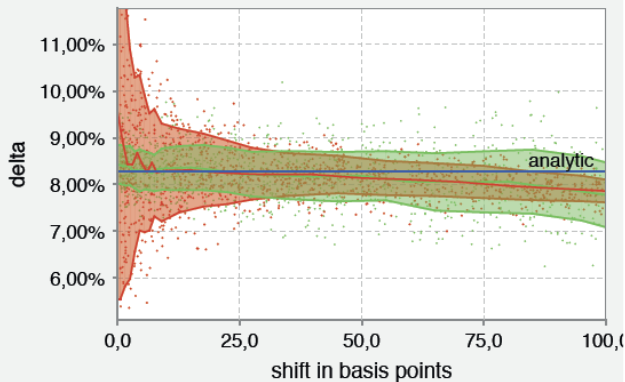
## Examples

- Each red point represents one simulation with naive bumping.
- Each green point is one simulation with proxy bumping.
- 1 standard deviation curves marked
- Many shift sizes shown.
- Graphs by Christian Fries (THANKS!)

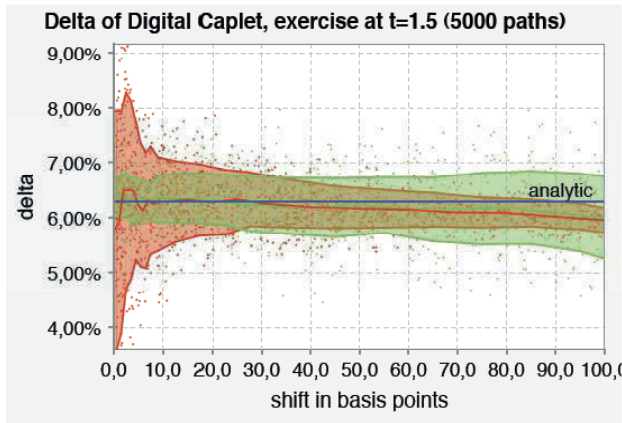
# Digital Caplet 1

partial proxy  
(1-dim LIBOR constant)

Delta of Digital Caplet, exercise at  $t=1.0$  (5000 paths)

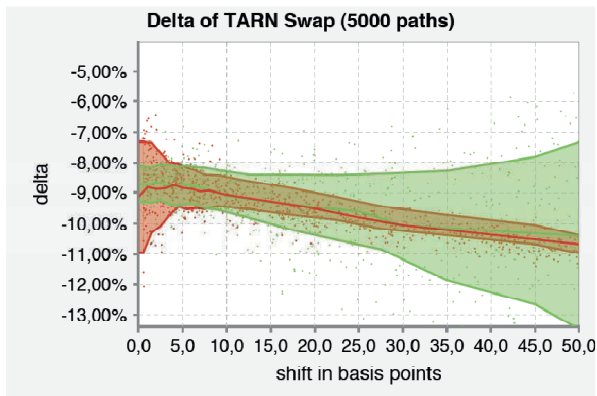


## Digital Caplet 2

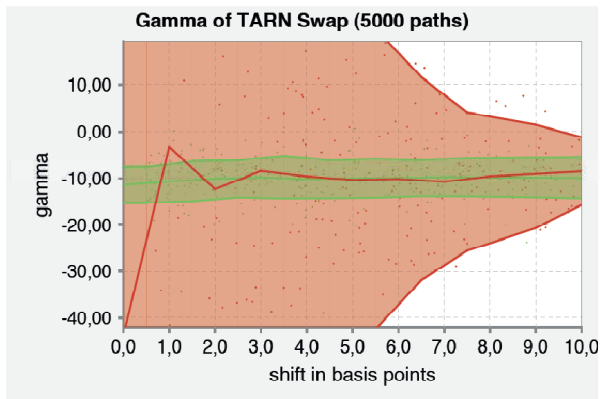


© 2006 Christian Fries - [www.christian-fries.de](http://www.christian-fries.de)

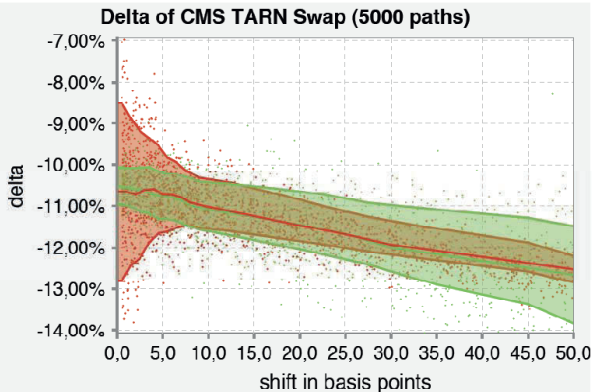
# TARN 1



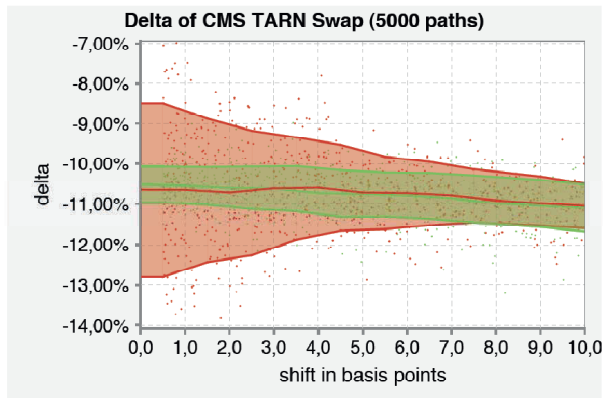
# TARN 2



# TARN 3

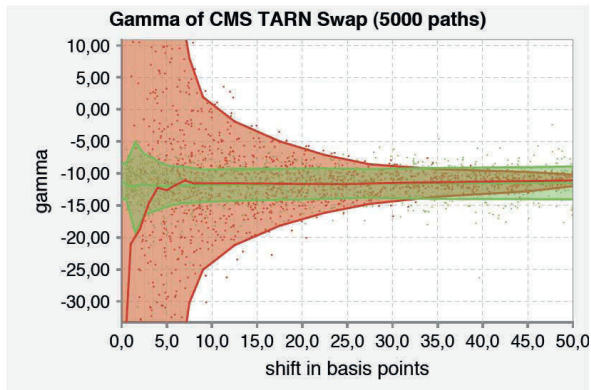


# TARN 4

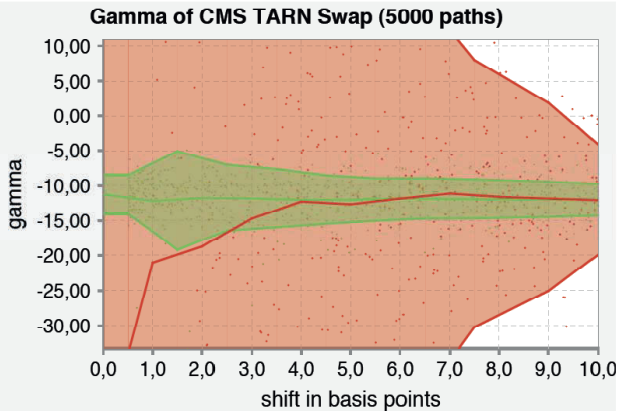




# TARN 5



# TARN 6



## Conditional analytic method

- The partial proxy method does not particularly well for vegas when
  - there is a very short time to first reset
  - the next coupon is discontinuous
- This will happen during the lifetime of any discontinuous product so we need to be able to cope with it.

## Analytic cash-flows

- Although path-dependents are very complicated and do not have closed-form solutions.
- The next cash-flow generally does!

## TARN trigger behaviour

- According to a reference rate, different behaviour occurs at each important time
- Rate below trigger, then we get a large fixed payment of remaining coupon and principal, and product terminates
- Rate above trigger, we get a smaller payment varying with level and product

# Conditional analyticity

- A cash-flow of the form

$$(Af_j + B)I_{f_j \leq C}P_{j+1},$$

is analytically evaluable.

- So do so.
- $A, B, C$  will be functions of the yield curve evolution up to time  $t_{j-1}$ .

# Smoothness

- We therefore value digital-type payments one step in advance.
- And subtract them from their natural step.
- The Black-value of a cash-flow will be a smooth function of its inputs even if digital so we have removed a discontinuity.

# Stratification

- After removing the smoothed term, we get zero and terminate if the rate is below the trigger.
- Otherwise, we get the continuation value.
- We can therefore restrict our sampling to the non-zero area provided we multiply by the probability of being in that area.



## Advantages of stratification

- Every path goes to the end.
- Every cash-flow non-zero.
- Everything smooth.
- We get a product of likelihood ratios but these are always less than 1 so this is not a problem.

# Implementing stratification

- Rewrite the pseudo-root for step so that the rate is only driven by one Gaussian,  $Z$ .
- The event that the rate is above a level  $H$  is then

$$\log f_0(s) + \mu + \sigma Z > \log H,$$

- Or

$$Z > \frac{\log H - \log f_0 - \mu}{\sigma} = Z_0.$$

# Implementing stratification II

- If  $Z = N^{-1}(u)$ .

- Then we want

$$u > p = N(Z_0).$$

- This is achieved by replacing  $u$  with

$$p + (1 - p)u.$$

- The likelihood ratio is  $1 - p$ .

## Running the path

- At each step, we gain the LR factor and the conditional analytic trigger values, we also gain the unconditional flows.
- Each cash-flow is multiplied by the product of likelihood ratios to get that far.
- Paths are much more similar to each other.
- Biggest plus is that there are no discontinuities.

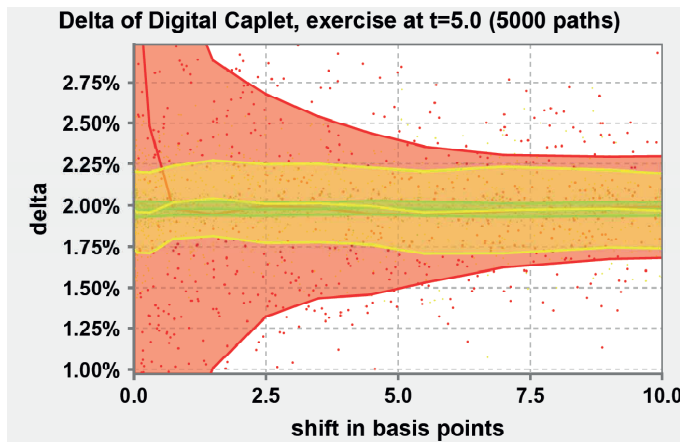
## Other rates

If the rate is not a forward LIBOR but instead a swap-rate then we can do an analytic approximation to get it to be approximately log-normal and proceed in the same way.

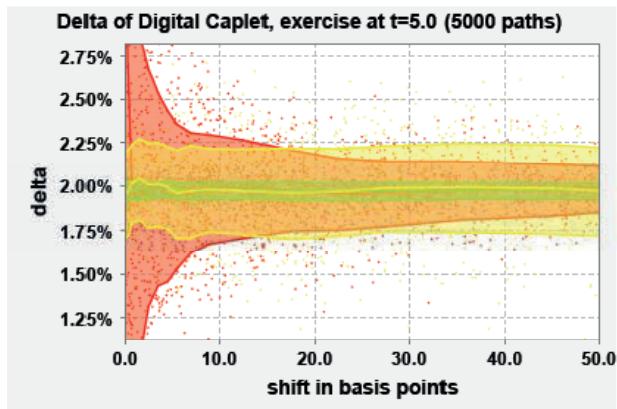
# Examples

- In what follows
  - Red is finite differences
  - Yellow is partial proxy
  - Green is conditional analytic
- Shift in basis points along bottom
- Width of coloured band denotes standard error

# CA: Digital Caplet 1

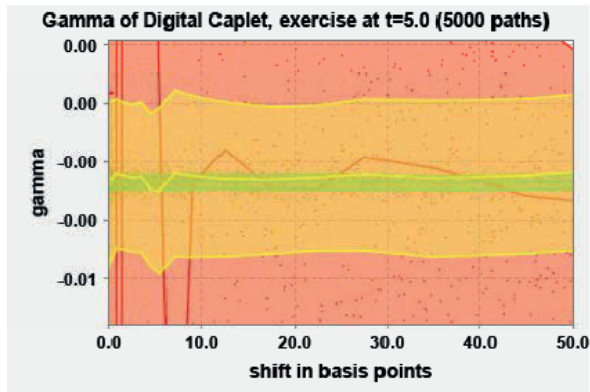


## CA: Digital Caplet 2

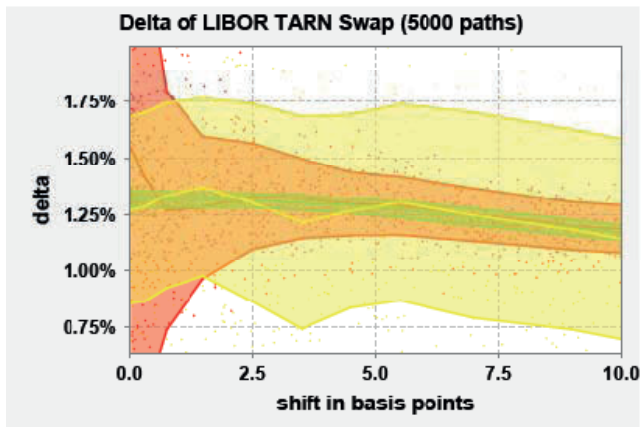




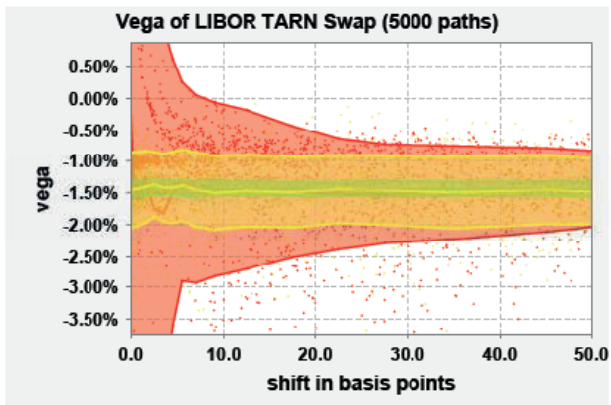
## CA: Digital Caplet 3



# CA: TARN 1



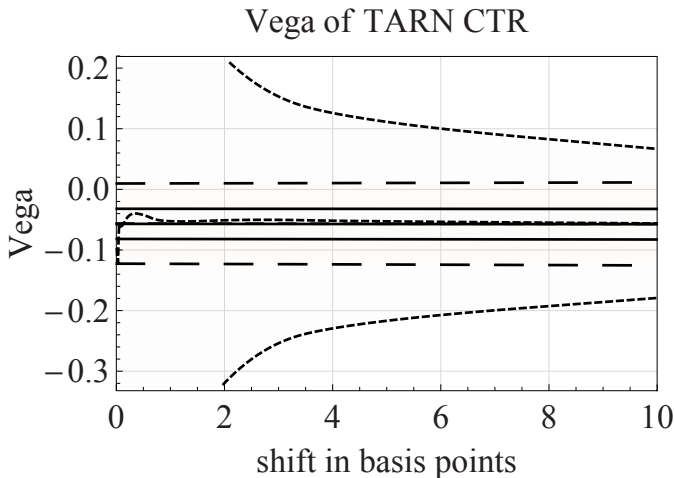
## CA: TARN 2



## Minimal partial proxy

- Partial proxy ensures that the trigger boundary is not crossed by constraining the triggering rate not to change.
- All we really need to do is to ensure that it does not cross the trigger level.
- Gives us more freedom.
- Replace Gaussian  $Z$  by  $aZ + b$  with  $a, b$  chosen to make the boundary not be crossed.
- One degree of freedom, so optimize to minimize the variance of the likelihood ratio.
- Results in lower variances for short term vegas.
- Not always better but always good enough.
- Relies on there only being one trigger level.

# TARN Close to reset using MPP vega



Solid line MPP, dashed PP, dotted finite differencing, 95th

percentiles

## Limit proxy methods

- In continuous case, finite differencing converges to the pathwise method as bump size goes to zero.
- Pathwise method is better because no need to run one simulation per Greek.
- In discontinuous case, it blows up.
- PP and MPP do not blow up and behave nicely for small bumps.
- Analyze their limit:
  - Can we get a pathwise method?
  - Can we get the adjoint speed ups?

## Limit proxy methods II

- Chan–Joshi 2011
- Pathwise partial proxy and pathwise minimal partial proxy.
- Small bump limit PP and MPP
- Low variance.
- Fast since one simulation for all Greeks and adjoint methods allow the same computational order.

# Market vegas

- The LMM pathwise method gives an effective method of computing pathwise vegas with respect to model parameters.
- In this case, the parameters are entries of the pseudo-root matrix.
- There are rather a lot of these. Approximately:

$$\frac{1}{2}n^2F.$$



# What a trader wants

- A trader wants the sensitivity to market observable quantities.
- This means the sensitivities to the implied volatilities of liquidly traded swaptions and caps.
- Our objective is therefore to translate model sensitivities into market sensitivities.

## Practical notion of vega

- How much does the price of an interest rate derivative change if a particular swaption (or cap) implied vol increases by 1% and all other swaption and cap implied vols remain fixed?
- Naive approach: bump the vol, recalibrate and see new price.
- Too noisy in practice.

## Closest calibration

- Rather than recalibrating, find the closest calibration with the desired change fixing the other instruments.
- Because uniquely defined and constrained, we do not have the noise problems.
- Requires us to define calibration and closest.

# Space of calibrations

- We evolve  $n$  rates across  $N$  steps with  $F$  factors. (We will take  $n = N$  for simplicity.)
- For each step,  $i$ , we use a pseudo-root of the covariance matrix,  $A_i$ .
- A calibration is therefore a vector of  $n$  pseudo-root matrices, each with  $nF$  entries.
- It is essentially a vector with  $nNF$  elements.

# Closeness

- We define the distance between two calibrations to be the ordinary Euclidean distance between these vectors.
- This will allow us to use elementary geometry!
- Note we are defining distance on pseudo-roots not covariances: so two calibration can be far apart whilst giving identical prices.

# Linearization

- We can rapidly price a swaption using the Hull–White formula for its implied vol.
- This involves covariances of rates, so we can differentiate it analytically to get the gradient of the implied vol as a function on the space of pseudo-roots.
- Similarly for caps.

# Vectors

- For each calibration instrument  $I_j$ , we therefore have a gradient vector  $v_j$ .
- To find the vega for instrument  $I_j$ , we therefore want a vector  $w_j$  such that

$$\langle w_j, v_j \rangle = 1\%,$$

$$\langle w_j, v_i \rangle = 0, \text{ for } i \neq j.$$

# Smallest vector

- We want the minimal perturbation so we take the vector  $w_j$  with smallest size.
- This can be found by simple geometry.
- We project  $v_j$  onto the orthogonal space to the span of the other vectors, and then scale to get

$$\langle w_j, v_j \rangle = 1\%.$$



# Dependence

- This methodology will fail if one instrument's gradients is approximately a span of other instruments' gradients.
- We then have to discard it and work with a smaller set of instruments.

# Too much freedom

- Since we have  $nNF$  pseudo-root elements, we have rather large vectors if  $n, N, F$  are large.
- E.g.

$$30 \times 30 \times 5 = 4500.$$

- We can reduce dimensionality by constraining elements to move in parallel

## Possible Constraints

- Each factor moves the same amount
- The bump to a rate is the same across several steps.
- For example, a parallel move of the volatility from 20 to 25 years makes sense since there is no data between those two time points.

# Vega computation

- 1 First decide calibrating instruments
- 2 Decide constraints for bumps
- 3 Find bumps that produce the appropriate vegas
- 4 Use these bumps with GG pathwise method

## Main References

- Fries–Joshi: Partial Proxy Simulation Schemes for Generic and Robust Monte-Carlo Greeks
- Fries–Joshi: Conditional Analytic Monte Carlo Pricing Scheme for Auto-Callable Products
- Chan-Joshi: Minimal Partial Proxy Simulation Schemes for Generic and Robust Monte-Carlo Greeks
- Chan-Joshi: Fast Monte-Carlo Greeks for Financial Products with Discontinuous Pay-Offs
- Glasserman-Staum: conditioning on one step-survival for barrier option simulations
- Joshi-Kwon: Monte Carlo Market Greeks in the Displaced Diffusion LIBOR Market Model
- Piterbarg: TARNs: Models, Valuation, Risk Sensitivities. Wilmott Magazine, 2004.

## Part XI

Kooderive: graphic cards and the LMM

# What is it?

- The most powerful processing unit in your PC is its graphics card.
- The most challenging use for a home PC is games.
- It is much easier to get a more powerful graphics card than a better CPU.
- GPGPU = general purpose graphics processing unit
- NVIDIA now produce graphics cards that don't do graphics!

# Why?

- GPUs are optimized for doing the same thing to every pixel on the screen
- Same instructions, different data
- This is very similar to Monte Carlo simulation
- Massive speed ups are possible eg 150x using two good GPUs versus single-threaded CPU code for the LMM in realistic cases
- It becomes possible to run 1 million paths, 32 rates, 32 steps, 5 factor model in less than a second with scrambled QMC



# What is different?

- Coding is as much about memory use and layout as about logic.
- 5 different sorts of memory to worry about.
- Many applications are *bandwidth bound* rather than *compute bound*. That is the time spent getting data on and off the graphics card takes longer than the computations.
- This is not an issue for Monte Carlo pricing, however, since we only need to pass in the calibration parameters and pass out the final price and standard error, and possibly some Greeks.

# CUDA

- CUDA = compute unified data architecture
- extension to C/C++ for writing GPU code integrated with CPU code,
- new compiler NVCC
- compiler/toolkit/SDK all freely downloadable from NVIDIA
- proprietary language unlike OpenCL which is the main competitor
- integrates with Visual Studio
- comes with really good free profiler.

# Threads and blocks

- GPU routines are called *kernels*
- Each one may have 10,000 or more threads.
- Threads are divided into *blocks*.
- Typically, 128 or 256 threads per block.
- Maybe 440 or more blocks.
- Each thread will do 1 to 10 paths for our simulation.

# Floating point arithmetic

- Early gpus did not support doubles.
- Later gpus support doubles but slow.
- So we generally work with floats.
- This is not a problem for MC but we do need to be aware of floating point error.

# Divergent branches

- We have 1000's of threads working in groups.
- These groups, or *warps*, have 16 or 32 members.
- The group must process the same instructions.
- Divergent branches mean that the time taken is the sum of the two branches.
- So minimize branching.

# Memory types

- host : CPU memory
- global: main memory on GPU, has odd properties for read and write
- constant: small amount of memory on GPU, writable by CPU, quickly readable by GPU
- shared: small amount of memory on GPU, shared between different threads in a block, supposedly very fast
- texture: a way of caching global memory to speed up access

# Global memory

- This is our main sort of memory, we may have 3 GB.
- Copying from CPU to GPU and back can take longer than generating results, but only if we want large data transfers.
- Speed of access on the GPU varies tremendously according to how its done.
- Coalesced access means thread  $i$  accesses memory location  $I + i$ . This is fast.
- Uncoalesced access is slow.
- We have to take this into account when designing our memory layouts.

# Textures

- A texture is a way of caching parts of global memory.
- Access is very fast.
- Read only : dangerous even to write to that part of global memory since the texture may still have the old value.
- New devices (2.0+) have “surface memory” that allows writes.



# Thrust

- Similar to STL but works on both device and host.
- Allows avoidance of memory commands simply by creating vectors.
- Contains standard algorithms written to work on GPU such as transform.
- Part of CUDA toolkit.

## Open source library

- [kooderive.sourceforge.net](http://kooderive.sourceforge.net),
- GPL 3.0 – you can do whatever you want with the code provided you extend the same rights to other people,
- no releases yet,
- code obtain from subversion repository, however,
- contains LMM pricer for path-dependent derivatives.

# Sublibraries

- gold : C++ CPU versions of all routines,
- kooderive: main gpu routines,
- cudaWrappers: wrappers around CUDA SDK/toolkit routines to make them easier to use,
- kooql: code for using kooderive and QuantLib together, eg use QuantLib for calibration, and then price using kooderive,
- Kootest: test routine for every kooderive kernel, runs against gold routine, and requires agreement up to floating point error.

# Gold

- C++ CPU static library,
- has no CUDA dependencies whatsoever,
- contains a full DDLMM path generator and pricer,
- useful in its own right,
- Code written to mimic GPU code rather than purely for speed, however.

# Collection of kernels

- Each kernel generally does one thing to a large number of paths, eg 16384.
- General philosophy is to keep each kernel as simple as possible.
- This facilitates testing and minimizes debugging time.
- Occasionally when everything works, write a bigger kernel that does multiple things to achieve extra speeds.
- Also allows easy slotting in and out of components without using virtual functions or templates.

# Kernels decomposition for LMM: LMMEvolutionRoutine

- 1 SobDeviceInts – generate Sobols integers
- 2 scramble\_main – scramble
- 3 thrust::transform to turn ints into normals
- 4 MultiDBridge – Brownian bridge the Sobols
- 5 correlated\_drift\_paths\_device – multiply by pseudo roots
- 6 LMM\_evolver\_pc\_gpu : generate forward rate paths
- 7 discount\_ratios\_computation\_main: generate discount factors
- 8 spot\_measure\_numeraires\_computation\_gpu\_main: generate discretely compounding money market account
- 9 forward\_rate\_extraction\_gpu\_main: extract the rates that the product depends on
- 10 cashFlowGeneratorCallerTARN: passed extracted rates to product (templated) and generate cash-flows
- 11 cashFlowDiscounting\_gpu\_main: discount cash-flows
- 12 thrust::reduce – average flows across paths to get price

# Batching

- Each path actually has quite a lot of data.
- 40 rates, 40 steps = 1600 floats =  $8 \times 1600$  bytes
- same amount again for discount ratios
- we also typically need the log rates for speed as well
- swap-rates, annuities
- cash-flows
- Even with 3GB, we can only store so many paths eg 65536 or 16384
- We therefore divide computation into say 32 batches of 16384.
- If we have more than one GPU, run a CPU thread for each GPU doing the same with different scrambling or skip in Sobol.

# Product specification

- Cash-flow specification templated on product class
- So easy to do different products.
- Extraction of relevant rates separate to product implementation.
- Thus can do CMS, LIBOR in arrears just by feeding in different rates



## Early exercise code

- In development...
- Store relevant aspect of paths rather than paths themselves
- This will allow many more paths at once. Eg 250,000
- Enough to have converged least-squares method

## 1 Computation of Discount ratios kernel

This is fairly simple mathematically, but it illustrates well the techniques used in Kooderive to carry out Monte Carlo simulations. Data passed in via textures, data passed out via a pointer to device data. Most of the kernels are very similar in structure. They simply have a different mathematical part.

```
// excerpt from LMM_Evolver_gpu.cu
```

```
// texture to use already generated rates
```

```
texture<float, 1, cudaReadModeElementType> tex_evolvedRates;
```

```
// texture storing location of first alive rate
```

```
texture<int, 1, cudaReadModeElementType> tex_alive;
```

```
// texture for accruals
```

```
texture<float, 1, cudaReadModeElementType> tex_taus;
```

```
__global__ // indicates that this is a device (GPU)
```

```
// function that can be called from the host (CPU)
```

```
void
```

```
discounts_ratios_computation_kernel_textures_all_steps(
```

```
    int taus_offset, // points to data in the texture
```

```
    int aliveIndex_offset, // points to data in the texture
```

```
    int rates_offset, // points to data in the texture
```

```
    float* discount_factors_global, // output data
```

```
    int paths, // total number of paths
```

```
    int n_rates, // number of rates
```

```
    int number_steps, // number of evolution steps
```

```
    int discLayerSize // the amount of data to jump per step
```

```
)
```

```
{
```

```
    int bx = blockIdx.x; // each thread has a block id
```

```
    int tx = threadIdx.x; // and a thread id,
```

```
// together they specify the thread's unique location in the grid
```

```
    int gx = gridDim.x; // how many blocks
```

```
    int bwidth = blockDim.x; // threads per block
```

```
    int width = gx*bwidth; // total number of threads
```

```

int pathsPerThread = 1 + (( paths -1)/width);
    // make each thread do enough so all paths are done

for (int l=0; l < pathsPerThread; ++l)
{

    int pathNumber = width*l + bwidth*bx+tx;
    // each thread does different paths

    if (pathNumber < paths) // generally try to pick inputs so this is always true
    {
        int pathOffset = pathNumber; // data's lowest dimension is paths
        // this is essential for coalescing
        // since we have defined pathNumber using + tx

        for (int s=0; s < number_steps; ++s)
        {
            int aliveIndex = tex1Dfetch(tex_alive, aliveIndex_offset+s);
            // restrict to rates that are still moving
            // use the texture to retrieve where they start

            float* discounts_location = discount_factors_global // start of data in cube
                +s*discLayerSize // highest dimension in cube
                +pathOffset // lowest dimension
                +aliveIndex*paths; // second lowest dimension

            *discounts_location = 1.0f;

            float df = 1.0f;

            int tex_rates_offset
                = rates_offset // start of texture data
                +pathOffset // path number lowest dimension
                +aliveIndex*paths // index of time amongst rate times, second lowest
                +s*paths*n_rates; // index of evolution step highest in cube

            for (int r=aliveIndex; r < n_rates; ++r)
            {

```

```

discounts_location+=paths;
    // next index location is paths further on

    float taur = tex1Dfetch(tex_taus,taus_offset+r);
    // use texture to get accrual

    float f = tex1Dfetch(tex_evolvedRates,tex_rates_offset);
    // use texture to retrieve forward rate

    df /= (1.0f+ taur*f); // the maths bit!
    *discounts_location = df; // write so can't use texture
    tex_rates_offset +=    paths; // move along the rates

```

```

    }

```

```

}

```

```

}

```

```

}

```

```

}

```

```

// routine that stands between main CPU code and GPU code
// _device means data is already on device

```

```

extern "C"

```

```

void

```

```

discount_ratios_computation_gpu(

```

```

    float* evolved_rates_device, // evolved rates for batch

```

```

    float* taus_device, // accruals

```

```

    const std::vector<int>& aliveIndices,

```

```

    int* alive_device, // same as previous entry but data on device

```

```

    int paths,

```

```

    int steps,

```

```

    int rates,

```

```

        float* discounts_device, // for output
        bool allStepsAtOnce // whether to invoke one kernel per step
    )
{

    const int threadsperblock = 512; // could be played with to suit given GPU
    const int maxBlocks = 65535; // a GPU characteristic

    // Set up the execution configuration
    dim3 dimGrid;
    dim3 dimBlock;

    // these data could be 3d objects but we treat them as 1d

    dimGrid.x = 1+(paths-1)/threadsperblock;
    // pick size that minimizes number of paths per thread

    if (dimGrid.x > maxBlocks) // make sure configuration OK
        dimGrid.x=maxBlocks;

    // Fix the number of threads
    dimBlock.x = threadsperblock;

    // CUDA timing code, more accurate than clock()
    unsigned int hTimer;
    double time;
    cutilCheckError(cutCreateTimer(&hTimer));
    cutilCheckError(cutResetTimer(hTimer));
    cutilCheckError(cutStartTimer(hTimer));


    // set up textures
    // we have to tell the texture what data to represent and cache

    cudaChannelFormatDesc channelDesc = cudaCreateChannelDesc(32, 0, 0, 0, cudaChannelFormatKindFloat);

```

```

// set texture parameters
tex_taus.addressMode[0] = cudaAddressModeWrap;
tex_taus.addressMode[1] = cudaAddressModeWrap;
tex_taus.filterMode = cudaFilterModeLinear;
tex_taus.normalized = false;    // access with normalized texture coordinates

// Bind the array to the texture, this is the important bit
// note that the textures are declared at the start of the file
cudaBindTexture( NULL, tex_taus, taus_device, channelDesc);

// set texture parameters
tex_evolvedRates.addressMode[0] = cudaAddressModeWrap;
tex_evolvedRates.addressMode[1] = cudaAddressModeWrap;
tex_evolvedRates.filterMode = cudaFilterModeLinear;
tex_evolvedRates.normalized = false;    // access with normalized texture coordinates

// Bind the array to the texture
cudaBindTexture( NULL, tex_evolvedRates, evolved_rates_device, channelDesc);


tex_alive.addressMode[0] = cudaAddressModeWrap;
tex_alive.addressMode[1] = cudaAddressModeWrap;
tex_alive.filterMode = cudaFilterModeLinear;
tex_alive.normalized = false;    // access with normalized texture coordinates

// Bind the array to the texture
cudaBindTexture( NULL, tex_alive, alive_device, channelDesc);


// textures are now set up

// errors can happen at strange times so make sure there are none before proceeding...

CUT_CHECK_ERR("discount_ratios_computation_gpu_execution_failed_before_entering_kernel\n");


int offset=0;

if (allStepsAtOnce)
{

```

```

    int discLayerSize = paths*(rates+1); // there is 1 more rate time than rate

// here is where we call the GPU routine
// the <<<dimGrid , dimBlock >>> is where we specify the blocks and threads per block
    discounts_ratios_computation_kernel_textures_all_steps<<<dimGrid , dimBlock >>>( offset ,
        offset ,
        offset ,
        discounts_device ,
        paths ,
        rates ,
        steps ,
        discLayerSize
    );
}
else
{

    for (int s=0; s < steps; ++s)
    {

        discounts_ratios_computation_kernel_textures<<<dimGrid , dimBlock >>>(offset , // texture offset
            aliveIndices[s] ,
            s*paths*rates ,
            discounts_device+s*paths*(rates+1),
            paths ,
            rates );

        cutiSafeCall(cudaThreadSynchronize());
    }
}

// no guarantee that the kernel is finished yet so wait until it has

    cutiSafeCall(cudaThreadSynchronize());

    CUT_CHECK_ERR("discount_ratios_computation_gpu_execution_failed_after_entering_kernel\n");

// the texture is no longer needed so unbind it
// alternate approach is to use a wrapper object

```

```
// that unbinds it when the scope left
    cudaUnbindTexture(tex_taus);

    cutilCheckError(cutStopTimer(hTimer));
    time = cutGetTimerValue(hTimer);
    std::cout << "_discounts_computation_"
        << time * 1E-3 << std::endl;

}
```

## 2 Cash-flow generation

The cash-flow generation code is a little different since it is templated. Here we take excerpts from `cashFlowGeneration_product_gpu.cu` and `cashFlowGeneration_gpu.h`

```
//
//          cashFlowGeneration_product_gpu.cu
//
//

// (c) Mark Joshi 2010
// This code is released under the GNU public licence version 3
#include "cashFlowGeneration_product_gpu.h"
#include "cashFlowGeneration_gpu.h"

// product specific data is passed in via the texture tex_aux_data
// pays  $(K - mf)_+ \tau$ 
// data total coupon, K, m, fixed rate accruals, floating rate accruals
class TARN
{

private:
    int offset_; // for texture
    int numberSteps_; // number of steps in simulation
```



```

    int index_; // current step

    float totalCoupon_; // target coupon
    float strike_;      // strike for inverse floater
    float multiplier_; // multiplier on rates in coupon

    float couponPaid_; // coupon paid so far
    bool done_;        // has product terminated

public:

    __device__ // indicates that this constructor can be called on the device
    // so object will be created by a kernel
    TARN(int offset, int numberSteps)
    {
        // set up member data from texture
        numberSteps_ = numberSteps;
        totalCoupon_ = tex1Dfetch(tex_aux_data, offset); // the total coupon during the product's
                                                         // fixed life time at offset

        offset_ = offset;
        ++offset_;
        strike_ = tex1Dfetch(tex_aux_data, offset_); // the strike, K, of the inverse
                                                         // floater coupons at offset plus 1

        ++offset_;
        multiplier_ = tex1Dfetch(tex_aux_data, offset_); // the rate multiplier, m,
                                                         // of the inverse floater at offset plus 1

        ++offset_;
        newPath();
    }

    __device__ void newPath()
    {
        // reinit data at start of path
        index_ = 0;
        couponPaid_ = 0.0f;
        done_ = false;
    }

    __device__ int getIndex()
    {

```

```

    return index_;
}

// this class will be a template parameter
// so signature fixed by cash-flow generation routines
// note absence of timing data for cash-flows
// the payment schedule is not relevant here
// it would be specified elsewhere independently
// This routine will be called from within a kernel
__device__ bool getCashFlows(float& flow1, // size of first cash-flow for this step
                             float& flow2, // size of second cash-flow for this step
                             float rate1,  // these rates could
                             float rate2,  // be forwards or swap-rates
                             float rate3,  // same class regardless
                             float* allForwardRates,
                             float* allDiscountRatios,
                             int paths )
{

    if (done_)
        return done_;

    float accrual2 = tex1Dfetch(tex_aux_data, offset_+index_+numberSteps_);
    flow2 = accrual2*rate2;

    if (index_ +1 == numberSteps_)
    {
        flow1 = couponPaid_-totalCoupon_ ;
        done_ = true;
        return done_;
    }

    float accrual1 = tex1Dfetch(tex_aux_data, offset_+index_);

    float uncappedFlow = (strike_-multiplier_*rate1)*accrual1;

    if (uncappedFlow <0.0f)
        uncappedFlow =0.0f;

```

```

float newTotalCoupon = couponPaid_ + uncappedFlow;
if (newTotalCoupon >= totalCoupon_)
{
    done_ = true;
    flow1 = couponPaid_ - totalCoupon_ ;
    return done_;
}

flow1 = - uncappedFlow;
couponPaid_ = newTotalCoupon;

++index_;

return done_;

}

};

extern "C"
void cashFlowGeneratorCallerTARN(float* genFlows1, float* genFlows2,
                                float* aux_data, int paths, int numberSteps,
                                float* rates1,
                                float* rates2,
                                float* rates3,
                                float* forwards,
                                float* discRatios)

{
    // we call a routine that is templated on the product type
    cashFlowGenerator<TARN>(genFlows1, genFlows2, aux_data, paths, numberSteps,
                            rates1, rates2, rates3, forwards, discRatios);
}

```

Now the routine that uses the TARN object.

```

//
//
//
//
//
//
// (c) Mark Joshi 2010
// This code is released under the GNU public licence version 3

/*
Routines are in the header file since they are templated on the product.
*/

#ifndef CASH_FLOW_GENERATION_H
#define CASH_FLOW_GENERATION_H

// texture used for product data
texture<float, 1, cudaReadModeElementType> tex_aux_data;

template<class T>
__global__ void cashFlowGenerator_kernel(float* genFlows1, // outputs of one leg
                                         float* genFlows2, // outputs of other leg
                                         int texture_offset, // offset into auxiliary data
                                         int paths, // paths in batch
                                         int numberSteps, // steps in the sim
                                         float* rates1, // first set of rates for product pay-off
                                         float* rates2, // second set of rates for product pay-off
                                         float* rates3, // third set of rates for product pay-off
                                         float* forwards, // all forward rates
                                         float* discRatios // all discount ratios
                                         )
{
// usual identify the path from thread and block data stuff
    int bx = blockIdx.x;
    int tx = threadIdx.x;

    int gx = gridDim.x;
    int bwidth = blockDim.x;

```

```

int width = gx*bwidth;

int pathsPerThread = 1 + (( paths -1)/width);

// create an object of type T in device memory
T obj(texture_offset,numberSteps);

for (int l=0; l < pathsPerThread; ++l)
{
    int pathNumber = width*l + bwidth*bx+tx;
    int dataOffset = pathNumber;
    bool done = false;
    if (pathNumber < paths)
    {

        obj.newPath(); // place object at start of path

        for (int i =0; i < numberSteps; ++i)
        {
            float flow1=0.0;
            float flow2=0.0;
            if (!done)
            {

                float rate1= rates1[dataOffset];
                float rate2= rates2[dataOffset];
                float rate3= rates3[dataOffset];

                // pass individual rates into the object and get the flows
                done = obj.getCashFlows(flow1 ,flow2 ,rate1 ,rate2 ,rate3 ,
                                         forwards ,discRatios ,paths);

            }

        }

        // write the flows out again
        genFlows1[dataOffset] = flow1;
        genFlows2[dataOffset] = flow2;
        dataOffset += paths;
    }
}

```

```

    }
}

// host routine that calls the kernel

template<class T>
void cashFlowGenerator(float* genFlows1, float* genFlows2, float* aux_data,
                      int paths, int numberSteps,
                      float* rates1, float* rates2, float* rates3,
                      float* forwards, float* discRatios)
{

    const int threadsperblock = 64;
    const int maxBlocks = 65535;

    // Set up the execution configuration
    dim3 dimGrid;
    dim3 dimBlock;

    dimGrid.x = 1+(paths-1)/threadsperblock;

    if (dimGrid.x > maxBlocks)
        dimGrid.x=maxBlocks;

    // Fix the number of threads
    dimBlock.x = threadsperblock;

    cudaChannelFormatDesc channelDesc = cudaCreateChannelDesc(32, 0, 0, 0, cudaChannelFormatKindFloat);

    // set texture parameters
    tex_aux_data.addressMode[0] = cudaAddressModeWrap;
    tex_aux_data.addressMode[1] = cudaAddressModeWrap;
    tex_aux_data.filterMode = cudaFilterModeLinear;
    tex_aux_data.normalized = false;    // access with normalized texture coordinates
    cudaBindTexture( NULL, tex_aux_data, aux_data, channelDesc);

    int offset=0;

    cashFlowGenerator_kernel<T><<<dimGrid , dimBlock >>>(genFlows1,

```

```

        genFlows2 ,
        offset ,
        paths ,
        numberSteps ,
        rates1 ,
        rates2 ,
        rates3 ,
        forwards ,
        discRatios );

    cudaUnbindTexture ( tex_aux_data );

}
#endif

```

Clearly, we could write an appropriate class for any path-dependent LIBOR product and proceed similarly. Note typical aspects of all this code. These are my choices rather than necessities.

- Input data uses textures.
- Output data uses global memory organized with pathnumber in the lowest dimension.
- Every kernel does one simple thing.
- Little use of shared memory or constant memory.
- All path by path work is done on the device.
- Flexibility achieved via passing different data in or by template parameter.
- Each path is handled by a single thread.
- No communication between threads.
- Data is king not objects!

